

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	14 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 1 — Foundations and First Deployment

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 1

How to read this chapter

Every topic below follows the same 16-point structure. That consistency is deliberate: once you know the shape, you can find "the security bit" or "the troubleshooting bit" for any topic without hunting.

The classroom gives you the mental model. **This manual gives you the depth** — the high availability, disaster recovery, performance and security material that would take three days to deliver as lecture. Read the relevant section the evening before you need it, and again a month after the course when you meet the topic in production.

Section	Answers
1. What it is	Plain English, no jargon in the first sentence
2. Why it exists	What was painful before
3. Business problem	In AxisPay terms, with money attached
4. How it works	The mechanism, in order
5. Internal architecture	Components and responsibilities
6. Component interactions	Who calls whom
7. Enterprise example	How a real payments platform uses it
8. Real-world analogy	One analogy — and its limits
9. Best practices	Rules, each with a reason
10. Common mistakes	With the symptom each produces
11. Security	Threat → control → residual risk
12. Performance	What gets slow, at what scale
13. High availability	Surviving node and zone loss
14. Disaster recovery	What to back up, RTO/RPO
15. Monitoring	Specific metrics and thresholds
16. Troubleshooting	Symptom → cause → command → fix
+	Interview questions with model answers

1.1 The declarative model and the reconciliation loop

1. What it is

You do not tell Kubernetes what to do. You tell it what you want, and it works continuously to make that true.

You write down a description of the world you want — "three copies of the payment service should be running" — and hand it to the cluster. Something inside the cluster then compares that description against what is actually running, notices any difference, and acts to close the gap. It does this forever, whether or not you are watching.

2. Why it exists

Before this model, operations was a sequence of instructions someone had to execute: *start this process on that server, and if it dies, start it again*. The instruction was only as reliable as the person or script running it, and the moment reality drifted from intention — a crash, a reboot, a full disk — nothing corrected it until a human noticed.

The declarative model moves the intention into the system itself. Correction becomes automatic because it is the system's only job.

3. The business problem

At 02:14 a node in AxisPay's cluster runs out of memory. The kernel's OOM-killer terminates the single `payment-service` process.

Without reconciliation: card authorisations fail for 41 minutes until an engineer wakes, reads the alert and restarts the process. Kalahari Coffee Roasters lose R38,000 of Saturday trade. Three merchants breach their contractual uptime SLA.

With reconciliation: a controller observes that 2 pods exist where 3 should, and creates a replacement. Elapsed time: about eight seconds. Nobody is paged. The engineer reads about it over breakfast.

4. How it works

Every object in Kubernetes has two halves:

Half	Meaning	Who writes it
<code>spec</code>	Desired state — what you want	You
<code>status</code>	Actual state — what is	A controller

A controller is a program that runs this loop, forever:

1. OBSERVE read spec and status from the API server
2. DIFF compare them
3. ACT if they differ, take one step towards spec
4. REPEAT

That is the whole idea. Everything else in this course is a variation on it.

5. Internal architecture

Controllers do not poll. They use a **watch** — a long-lived streaming connection to the API server that pushes changes as they happen. Each controller maintains a local cache (an *informer*) and a work queue.

```

API server —watch→ Informer → Work queue → Worker → API server
                  (cache)      (dedup,      (reconcile)  (write)
                              rate limit)

```

Three properties matter in practice:

- **Level-triggered, not edge-triggered.** A controller acts on the *current state*, not on the event that woke it. If it misses an event, the next sync corrects anyway. This is why Kubernetes is resilient to controllers restarting.
- **Idempotent.** Reconciling twice produces the same result as reconciling once.
- **Eventually consistent.** There is always a window where `status` lags `spec`. That window is why `kubectl get` immediately after `kubectl apply` shows nothing yet.

6. Component interactions

Deleting one pod from a 3-replica Deployment:

```

you           kubectl delete pod
API server    pod marked for deletion, status written to etcd
ReplicaSet    watch fires → observes 2 running, spec says 3
controller    creates a new Pod object (spec.nodeName is EMPTY)
scheduler     watch fires → filters and scores nodes → binds pod to axispay-m02
kubelet(m02)  watch fires → pod assigned to me → calls containerd
containerd    pulls (cached) image, starts container
kubelet       writes status: Running back to the API server

```

Five components. None called another. Each watched the API server and did one small thing.

7. Enterprise example

A tier-1 bank runs 4,000 microservice replicas across three availability zones. Nodes are replaced continuously — patched, rotated for compliance, reclaimed by the cloud provider. On a normal day, several hundred pods are destroyed and recreated. No human is involved in any of it, and no ticket is raised. The reconciliation loop absorbs the churn.

The operational discipline that makes this safe is that every workload declares what it needs (replicas, resources, disruption budget) and the platform enforces it. Nobody logs into a node.

8. Real-world analogy

A thermostat. You set 21°C — that is `spec`. It reads the room temperature — that is `status`. If the room is 19°C it turns on the heating. It does not execute a script called "heat the room for twelve minutes"; it continuously compares and corrects.

Where the analogy breaks: a thermostat controls one variable with one actuator.

Kubernetes runs hundreds of independent controllers over thousands of objects, and controllers can conflict — two controllers both trying to own the same pod will fight. That is why `spec.selector` overlap between Deployments is a genuine (and confusing) production bug.

9. Best practices

Practice	Reason
Keep manifests in Git and apply from there	The repository becomes the real source of truth; the cluster becomes a projection of it
Use <code>kubectl apply</code>, not <code>create</code>	<code>apply</code> is idempotent and records intent in an annotation, so re-running is always safe
Never edit live objects with <code>kubectl edit</code> in production	Your change is invisible to Git and will be silently reverted by the next apply
Treat <code>status</code> as read-only, always	Writing status is a controller's job; if you find yourself wanting to, you want a different object
Set <code>metadata.annotations["kubernetes.io/change-cause"]</code>	<code>kubectl rollout history</code> becomes readable during an incident
Expect eventual consistency	Poll with <code>kubectl rollout status</code> rather than asserting immediately after apply

10. Common mistakes

Mistake	Symptom
Using <code>kubectl scale</code> and forgetting the YAML	Replica count silently reverts on the next <code>apply</code> . "It worked yesterday."
Expecting <code>apply</code> to be synchronous	Script checks for pods immediately, finds none, and reports failure
Assuming deleting an object stops the controller	Deleting a <i>pod</i> does nothing lasting; you must delete the <i>controller</i>
Editing a field the controller owns	Your change is overwritten within seconds, apparently at random
Two Deployments with overlapping selectors	Pods flap between owners; replica counts oscillate. Very hard to diagnose.

11. Security considerations

Threat	Control	Residual risk
Anyone who can write <code>spec</code> controls the workload	RBAC on verbs (<code>create</code> , <code>update</code> , <code>patch</code>) per resource per namespace	A compromised CI service account can still deploy anything within its grant
A malicious manifest schedules a privileged pod	Pod Security Admission at namespace level (Day 5)	Admission runs at write time; existing pods are unaffected
Controllers hold broad permissions by design	Audit <code>ClusterRoleBinding</code> s; prefer namespaced <code>Role</code> s	The controller-manager itself is necessarily privileged
<code>kubectl edit</code> bypasses code review	Enforce GitOps; restrict <code>update</code> in production namespaces	Break-glass access must still exist for incidents

12. Performance considerations

- **Watch, not poll.** A controller that polls the API server every second across 10,000 objects will saturate it. All in-tree controllers use watches; if you write an Operator, use the informer framework rather than a loop.
- **Resync period.** Informers do a full resync periodically (commonly 10 minutes) to correct any missed events. Shorter resyncs increase API server load for little benefit.
- **etcd is the ceiling.** Every write goes through Raft consensus. Clusters degrade at roughly 5,000+ nodes or very high object churn; the symptom is rising `etcd_disk_wal_fsync_duration_seconds`.
- **Leading indicator:** `apiserver_request_duration_seconds` p99 climbing above ~1s for `LIST` operations means something is listing too much, too often.

13. High availability

The control plane should run three or five API server, scheduler and controller-manager instances across failure domains. Scheduler and controller-manager use **leader election** — only one is active; the others stand by and take over within seconds.

Critically: if the entire control plane is down, running workloads keep running.

Kubelets continue managing their existing pods and containers keep serving traffic. What stops is *change*: no new pods, no rescheduling, no scaling, no self-healing. This distinction matters enormously during an incident — a control-plane outage is serious but is not, by itself, a customer-facing outage.

14. Disaster recovery

What to back up	etcd. It contains every object. Nothing else in the control plane holds state.
How	<code>etcdctl snapshot save</code> on a schedule, shipped off-cluster, encrypted at rest
RPO	Your snapshot interval. Hourly is common; 15 minutes for regulated workloads.
RTO	15–60 minutes to restore etcd and verify, on a practised team
The rule	A backup you have never restored is not a backup. Test the restore quarterly.
Better still	Keep every manifest in Git. Then etcd restore is a convenience, not a lifeline — you can rebuild from source.

15. Monitoring

Metric	Why	Alert at
<code>kube_deployment_status_replicas_available</code> vs <code>..._spec_replicas</code>	The gap the loop is failing to close	Gap persists > 5 min
<code>workqueue_depth</code> (per controller)	Controller falling behind	Sustained rise
<code>workqueue_adds_total</code> rate	Reconciliation churn — often a fight between controllers	Unexplained spike
<code>apiserver_request_duration_seconds</code> p99	Control-plane health	> 1s
<code>etcd_server_leader_changes_seen_total</code>	Control-plane instability	Any increase

16. Troubleshooting

Symptom	Likely cause	Command	Fix
Applied, nothing happened	Wrong namespace or context	<code>kubectl config view --minify</code>	Set context/namespace
Object reverts after edit	A controller owns that field	<code>kubectl get <obj> -o yaml grep -A5 ownerReferences</code>	Edit the owner, not the object
Replica count oscillates	Two controllers with overlapping selectors	<code>kubectl get deploy -o json jq '.items[].spec.selector'</code>	Make selectors disjoint
status never updates	Controller unhealthy	<code>kubectl -n kube-system logs -l component=kube-controller-manager</code>	Restart or investigate
Everything stuck, nothing scheduling	Scheduler down or all nodes tainted	<code>kubectl get pods -n kube-system;</code> <code>kubectl describe node</code>	Restore scheduler; check taints

Interview questions

- What is the difference between imperative and declarative management, and why does it matter operationally?** *Imperative specifies the steps; declarative specifies the outcome and a controller determines the steps continuously. Operationally: declarative is idempotent, self-correcting, and reviewable in version control. The practical consequence is that drift is corrected automatically rather than discovered by a customer.*
- A pod belonging to a Deployment is deleted. Trace what happens.** *Actual state drops to $N-1$. The ReplicaSet controller observes the difference and creates a new Pod object with no `nodeName`. The scheduler filters and scores nodes and binds the pod. The kubelet on that node sees a pod assigned to it and instructs the container runtime to start it, then writes status back. Around eight seconds end-to-end.*
- The entire control plane is down. What still works?** *Existing pods keep running and serving traffic — kubelets do not need the control plane for steady state. What stops is change: no scheduling, no scaling, no self-healing, no `kubectl`. Serious, but not automatically a customer-facing outage.*
- Why are Kubernetes controllers level-triggered rather than edge-triggered?** *(senior) Level-triggered means acting on current state rather than on the event that triggered the wake-up. This makes the system resilient to lost events, controller restarts and network partitions — the next reconcile corrects regardless. Edge-triggered systems must guarantee event delivery, which is far harder to build correctly.*
- You update a field on a live object and it reverts within seconds. Why?** *(senior) A controller owns that field and is reconciling towards its own `spec`. Check `ownerReferences` to find the owner and change it there. A common example is editing a pod that belongs to a ReplicaSet.*

1.2 Cluster architecture

1. What it is

A Kubernetes cluster is a set of machines split into two roles: a **control plane** that decides what should happen, and **nodes** that run the actual workloads.

2. Why it exists

Separating decision-making from execution allows either to fail independently and be scaled independently. The control plane can be made highly available without touching workloads; nodes can be added, drained and destroyed without the cluster losing its mind.

3. The business problem

AxisPay must survive losing a machine without losing payments. That requires something with a global view — which nodes exist, what is running, what should be — that is not itself running on the machine that failed.

4. How it works

The single most important structural fact: every component communicates only with the API server. The scheduler never calls the kubelet. The controller manager never calls etcd. Each component watches the API server for the objects it cares about and writes its results back.

This is what makes Kubernetes extensible: to add behaviour, you write another program that watches the API server. It is also why `kubectl` can observe everything — there is only one place anything happens.

5. Internal architecture

Control plane

Component	Single responsibility	Notes
kube-apiserver	The only door. Every request passes authentication → authorisation → admission.	Stateless; scale horizontally
etcd	The only source of truth. Distributed key-value store holding every object.	Raft consensus; needs odd member count
kube-scheduler	Decides where . Filters unsuitable nodes, scores the rest, binds the winner.	Writes only a node name. Never starts anything.
kube-controller-manager	Runs ~30 reconciliation loops — ReplicaSet, Deployment, Endpoint, Job, Node.	Leader-elected
cloud-controller-manager	Cloud integration: load balancers, routes, node lifecycle.	Absent on Minikube

Every node

Component	Responsibility
kubelet	Makes pods real on this node. Watches for pods assigned to it, drives the container runtime, runs your probes , reports status. Not a container itself.
kube-proxy	Programs Service routing into the kernel (iptables or IPVS). No proxy process in the request path.
Container runtime (CRI)	containerd — actually runs containers
Network plugin (CNI)	Calico — assigns pod IPs, enforces NetworkPolicy
Storage plugin (CSI)	Attaches and mounts volumes

Why this course insists on `--cni=calico`. Kubernetes does not implement networking; it defines the CNI interface. Minikube's default plugin provides connectivity but **does not enforce NetworkPolicy**. On a default cluster, every Day 4 security policy would apply cleanly and protect nothing — the worst possible outcome in a security module. CNI cannot be changed on a running cluster, which is why L1.1 makes you verify it on Monday.

6. Component interactions

See §1.1 point 6 — the `kubectl apply` trace is the canonical example.

Static pods. On Minikube and kubeadm clusters, the control-plane components themselves run as pods — but they are created by the kubelet directly from files in `/etc/kubernetes/manifests/`, not by the API server. That is the bootstrap answer to "what starts the API server if the API server creates everything?"

7. Enterprise example

A payments processor runs three API servers behind a load balancer across three availability zones, with a five-member etcd cluster on dedicated NVMe. Scheduler and controller-manager run in all three zones with leader election. Losing an entire zone costs one API server and one etcd member; the cluster continues without interruption. Control-plane nodes run *no* workloads — `node-role.kubernetes.io/control-plane:NoSchedule` keeps them clear.

8. Real-world analogy

An airport. The control tower (control plane) does not fly aircraft — it decides which runway each aircraft uses and when. The aircraft (nodes) do the flying. If the tower goes silent, aircraft already airborne continue safely; what stops is new departures and reassignments.

Where it breaks: an airport has one tower. A production cluster has three, any of which can take over instantly.

9. Best practices

Practice	Reason
Three or five etcd members, never even numbers	Raft needs a majority; four members tolerate the same single failure as three but cost more
Put etcd on fast, dedicated disks	etcd is fsync-bound. Disk latency is the number-one cause of control-plane instability.
Do not schedule workloads on control-plane nodes	A noisy pod must never be able to starve the API server
Spread control-plane replicas across failure domains	Otherwise HA is theatre
Enable audit logging	You cannot investigate what you did not record — and regulators will ask
Keep kubectl within one minor version of the server	Version skew produces confusing, silent failures

10. Common mistakes

Mistake	Symptom
Even number of etcd members	No improvement in fault tolerance; cluster loses quorum unexpectedly
etcd on network storage	Random control-plane latency spikes, leader elections, mysterious slowness
Believing the scheduler starts containers	Confusion when debugging <code>Pending</code> vs <code>ContainerCreating</code>
Ignoring the CNI choice	NetworkPolicies silently not enforced — the failure has no error message
Backing up nothing because "it's declarative"	True only if every manifest is really in Git. Most clusters have drift.

11. Security considerations

Threat	Control	Residual risk
etcd read = every Secret in the cluster, in plaintext	Encryption at rest; restrict etcd to control-plane nodes; mTLS between API server and etcd	Encryption keys must themselves be protected
Compromised API server = full cluster control	Strong authN (OIDC/certs), RBAC, admission control, audit logging	Cluster-admin remains cluster-admin
Kubelet API exposed without auth	<code>--anonymous-auth=false</code> , <code>--authorization-mode=Webhook</code>	Node compromise still yields local container access
Workloads on control-plane nodes	Taints plus admission policy	Break-glass exceptions

12. Performance considerations

- **API server** scales horizontally; it is stateless. Watch `apiserver_request_duration_seconds` and inflight-request limits.
- **etcd** does not scale horizontally for writes — more members means *more* consensus work. Scale vertically: faster disks.
- **Scheduler throughput** is around 100 pods/second by default. `percentageOfNodesToScore` trades placement quality for speed on large clusters.
- **Watch out for LIST storms**: a controller (or a badly written Operator) listing all pods every few seconds is the most common cause of API server saturation.

13. High availability

Component	HA mechanism	Failure behaviour
API server	N replicas behind a load balancer	Stateless; any instance serves
etcd	Raft, 3 or 5 members	Tolerates $(N-1)/2$ losses; below quorum it goes read-only
Scheduler	Leader election	Standby takes over in ~15s
Controller manager	Leader election	Same
kubelet	One per node, not HA	Node marked <code>NotReady</code> after ~40s; pods evicted after the toleration period (default 300s)

14. Disaster recovery

Scenario	Recovery
One etcd member lost	Remove, add a replacement, let it sync. No downtime.
etcd quorum lost	Restore from snapshot to a single member, then re-add. This is the real disaster. Practise it.
Control plane entirely lost	Rebuild control plane, restore etcd, rejoin nodes. Workloads kept running throughout.
Full cluster lost	Rebuild from manifests in Git. This is why GitOps is a DR strategy, not just a workflow.

RTO/RPO for a practised team: RTO 30–60 minutes, RPO equal to your snapshot interval.

15. Monitoring

Metric	Threshold
<code>etcd_disk_wal_fsync_duration_seconds</code> p99	> 25 ms — investigate disk
<code>etcd_server_leader_changes_seen_total</code>	Any sustained increase
<code>etcd_server_has_leader</code>	0 = quorum lost. Page immediately.
<code>apiserver_request_duration_seconds</code> p99	> 1 s
<code>apiserver_current_inflight_requests</code>	Approaching the configured limit
<code>scheduler_pending_pods</code>	Sustained > 0 with capacity available
<code>kubelet_node_ready</code> (per node)	Any 0

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>kubectl</code> times out	API server down / LB misrouting	<code>curl -k https://<api>:6443/healthz</code>	Check API server pods and LB
Everything Pending , capacity available	Scheduler down	<code>kubectl -n kube-system get pods -l component=kube-scheduler</code>	Restart scheduler
Nothing self-heals	controller-manager down	<code>kubectl -n kube-system logs -l component=kube-controller-manager</code>	Restart
Node NotReady	kubelet down or CNI unhealthy	<code>kubectl describe node <n> → Conditions</code>	Check kubelet, CNI pods
Writes fail, reads work	etcd quorum lost	<code>etcdctl endpoint status --cluster</code>	Restore quorum
NetworkPolicies do nothing	CNI does not enforce them	<code>kubectl get pods -n kube-system -l k8s-app=calico-node</code>	Rebuild with a policy-capable CNI

Interview questions

- Name the control-plane components and give each one sentence.** (*junior*)
- Does the scheduler start containers?** *No. It writes a node name into the pod object and stops. The kubelet on that node starts the container. Getting this wrong makes Pending versus ContainerCreating impossible to reason about.*
- Why must etcd have an odd number of members?** *Raft requires a majority. Three tolerates one loss; four also tolerates only one — you pay for a member that buys nothing and adds consensus latency.*
- How would you design a control plane to survive an availability-zone failure?** (*senior*) *Three control-plane nodes across three zones, API servers behind a zone-aware load balancer, five etcd members spread so no zone holds a majority, leader election for scheduler and controller-manager, and control-plane nodes tainted against workloads.*
- Your NetworkPolicies are applied but traffic still flows. What is wrong?** (*senior*) *The CNI does not implement policy enforcement. Kubernetes accepts and stores the object regardless — enforcement is the plugin's job. Verify with a policy-capable CNI such as Calico or Cilium.*

1.3 Namespaces

1. What it is

A namespace is a named scope inside a cluster. Object names must be unique within a namespace, not across the cluster.

2. Why it exists

To let many teams, environments and applications share one cluster without colliding — and to give RBAC, quotas and policies something to attach to.

3. The business problem

AxisPay handles cardholder data. A QSA auditing the platform asks one question that determines the cost of the entire audit: **what is in scope?** Everything that stores, processes or transmits cardholder data must meet the full PCI-DSS control set.

A flat, unsegmented platform means everything is in scope — every service, every developer's access, every log. Namespace design is the first line of the segmentation argument.

4. How it works

Most objects are namespaced; some are not. The namespace is part of the object's identity and appears in its DNS name: `payment-service.axispay-core.svc.cluster.local`.

5. Internal architecture

AxisPay's zones:

Namespace	Zone	pci-scope	Contains
<code>axispay-edge</code>	DMZ	<code>false</code>	edge-gateway, auth-service
<code>axispay-core</code>	CDE	<code>true</code>	payment, merchant, customer, fraud, routing, ledger
<code>axispay-async</code>	processing	<code>true</code>	settlement, notification, audit, reporting
<code>axispay-data</code>	vault	<code>true</code>	PostgreSQL, Redis, RabbitMQ (<i>Day 3</i>)
<code>axispay-observability</code>	monitoring	<code>false</code>	Prometheus, Grafana, Loki (<i>Day 5</i>)

`axispay-edge` is `pci-scope: false` because it **transmits** but does not **store or process** cardholder data — and in this platform it only ever sees a token, never a card number. That is a real audit argument and it hinges entirely on the tokenisation boundary.

6. Component interactions

Namespaces are the selector target for:

- **NetworkPolicy** — `namespaceSelector: {matchLabels: {axispay.io/zone: edge}}` (Day 4)
- **RBAC** — a `RoleBinding` grants within one namespace (Day 5)
- **ResourceQuota / LimitRange** — enforced per namespace (Day 2)
- **Pod Security Admission** — enforced by namespace label (Day 5)

This is why the labels matter more than the names.

7. Enterprise example

A bank runs one cluster per environment and, within production, one namespace per service domain with a strict naming convention. Namespace creation is automated: creating `payments-prod` automatically provisions a default-deny NetworkPolicy, a ResourceQuota, a LimitRange, RBAC bindings to the owning team's group, and Pod Security labels. Nobody creates a namespace by hand — the guarantees come with it.

8. Real-world analogy

Floors in an office building. Each has its own room numbers, so "Room 3" exists on every floor without confusion, and each floor has its own door badge policy.

Where it breaks: in a building, the walls between floors are physical. In Kubernetes, **namespaces are not walls**. The network between them is wide open by default.

9. Best practices

Practice	Reason
Never use <code>default</code> for real workloads	It has no quota, no policy, and no clear owner
Label namespaces with zone, scope, owner and cost centre	Every later control selects on labels, not names
One namespace per trust boundary, not per micro-team	Too many namespaces makes policy unmanageable
Apply ResourceQuota to every namespace	One namespace should never be able to starve another
Automate namespace creation with its guarantees attached	Manual creation always forgets something
Set your context namespace	<code>kubectl config set-context --current --namespace=<ns></code> prevents an entire class of mistake

10. Common mistakes

Mistake	Symptom
Believing namespaces isolate the network	Flat pod network across all namespaces — an audit finding, discovered late
Deploying to <code>default</code> by accident	Objects "missing"; quotas and policies not applied
Using <code>ClusterRoleBinding</code> where <code>RoleBinding</code> was meant	Access granted in every namespace, including future ones
Putting environments in namespaces of one cluster	A cluster-wide failure takes production and staging together
Deleting a namespace to "clean up"	Deletes everything inside it, including PVCs and their data

11. Security considerations

Threat	Control	Residual risk
Lateral movement between namespaces	NetworkPolicy default-deny (<i>Day 4</i>)	Requires a policy-enforcing CNI
Over-broad RBAC	Prefer <code>Role</code> over <code>ClusterRole</code> ; review bindings	Cluster-scoped resources still need ClusterRoles
Privileged pods in a regulated namespace	Pod Security Admission <code>restricted</code> (<i>Day 5</i>)	Applies at admission only
Secret sprawl	Secrets are namespaced; restrict <code>get / list</code> per namespace	A namespace admin can read its Secrets

A namespace is not a security boundary. It is the handle you hang security boundaries on.

12. Performance considerations

Namespaces are free at runtime — they add no data-path cost. The cost is control-plane: thousands of namespaces each with quotas, policies and RBAC increases API server memory and watch load. Namespace *deletion* can be slow, because it must delete every contained object and wait for finalisers.

13. High availability

Namespaces are metadata in etcd; they inherit control-plane HA. They do **not** provide availability isolation — a namespace does not confine a failure. Use nodes, zones and PodDisruptionBudgets for that.

14. Disaster recovery

Trivially reproducible from manifests — a namespace is a few lines of YAML. **But deleting one destroys every object inside it, including PersistentVolumeClaims and therefore potentially the data.** Protect production namespaces with RBAC restricting `delete` on namespaces, and set PV reclaim policy to `Retain` for anything holding data.

15. Monitoring

Metric	Why
<code>kube_namespace_status_phase{phase="Terminating"}</code>	Namespace stuck terminating — almost always a finaliser
<code>kube_resourcequota used vs hard</code>	Approaching quota causes confusing scheduling refusals
Object count per namespace	Runaway creation (a bad Operator or CI loop)

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>namespaces "x" not found</code>	Not created, or typo	<code>kubectl get ns</code>	Apply the manifest
Objects land in default	Context not scoped	<code>kubectl config view --minify grep namespace</code>	Set the namespace
Namespace stuck Terminating	A finaliser is blocking	<code>kubectl get ns <n> -o json jq .spec.finalizers</code>	Remove the blocking resource first
Pods rejected on create	ResourceQuota exceeded	<code>kubectl describe quota -n <ns></code>	Raise quota or lower requests
Cross-namespace call fails	Used the short DNS name	<code>nslookup</code> from inside a pod	Use <code><svc>.<ns>.svc.cluster.local</code>

Interview questions

- Do namespaces isolate network traffic?** No. By default every pod can reach every other pod in any namespace. Namespaces isolate names, RBAC scope and quotas. Network isolation requires NetworkPolicy plus a CNI that enforces it.
- Name three resources that are NOT namespaced.** Node, PersistentVolume, StorageClass, ClusterRole, ClusterRoleBinding, CustomResourceDefinition, IngressClass, Namespace itself.

3. What is the practical difference between RoleBinding and ClusterRoleBinding?

(senior) A RoleBinding grants within one namespace. A ClusterRoleBinding grants across every namespace, including ones created later. Using the latter by mistake is one of the most common accidental privilege escalations.

4. How would you design namespaces for a PCI-regulated payments platform?

(senior) Segment by trust zone rather than by team: a DMZ holding only the edge, a cardholder data environment, an async processing zone, an isolated data tier, and observability. Label each with zone and PCI scope, then anchor NetworkPolicy, RBAC and Pod Security Admission to those labels. Minimise what is in scope — anything that only ever sees tokens can be argued out of the CDE.

1.4 Pods

1. What it is

A Pod is one or more containers that are always scheduled together on the same node, share one IP address, and live and die as a unit. It is the smallest thing Kubernetes will schedule.

2. Why it exists

Some containers must share a network namespace and a filesystem to function: an application and its log shipper, an application and a proxy sidecar, an application and an init container that prepares its data. A scheduler that only understood individual containers could not express "these must be co-located".

3. The business problem

AxisPay's `payment-service` needs to run somewhere, with an IP other services can reach, and with the ability to add a log-forwarding sidecar later without rewriting the application.

4. How it works

The kubelet creates a `pause` container first. It does nothing except hold the network and IPC namespaces open. Your containers then *join* those namespaces. That is why your container can crash and restart without the Pod losing its IP address.

Inside a Pod:

- **One IP.** All containers share it and reach each other on `localhost`.
- **One port space.** Two containers cannot both bind 8080.
- **Shared volumes.** Any container can mount them.
- **Separate filesystems** (except mounted volumes) and separate process spaces by default.

5. Internal architecture

Container types within a Pod:

Type	Runs	Purpose
<code>initContainers</code>	Sequentially, to completion, before app containers	Wait for a dependency, run a migration, fetch config (<i>Day 3</i>)
<code>containers</code>	Concurrently, for the Pod's life	Your application
Sidecar (<code>initContainer</code> with <code>restartPolicy: Always</code>)	Starts before app containers, runs alongside	Log shipper, proxy (<i>Day 5</i>)

Pod phases:

Phase	Meaning
Pending	Accepted, not yet running — being scheduled, image pulling, or volume mounting
Running	Bound to a node; at least one container running
Succeeded	All containers exited 0 and will not restart
Failed	All terminated; at least one failed
Unknown	Node unreachable

Running does not mean Ready. `Running` says the container process started. `Ready` says a readiness probe passed and the Pod may receive traffic. `kubectl get pods` shows `1/1` or `0/1` for exactly this reason, and the difference is where most Day 2 confusion lives.

6. Component interactions

```
scheduler  writes spec.nodeName
kubelet    sees a pod assigned to it
kubelet    CRI: create pause container → network namespace exists
kubelet    CNI: assign an IP to that namespace
kubelet    CSI: mount any volumes
kubelet    CRI: run initContainers, in order, to completion
kubelet    CRI: start app containers
kubelet    begins probing; writes status back to the API server
```

7. Enterprise example

A payment service Pod in production runs three containers: an init container that waits for the database and applies migrations; the application; and a sidecar that forwards logs and exposes metrics. All three share the Pod's IP and a volume for temporary files. The application does not know the sidecar exists — a deliberate separation that lets the platform team change log shipping without a single application release.

8. Real-world analogy

A shipping container on a ship. Everything inside travels together, arrives together and is unloaded together. You do not ship half a container to Durban and half to Cape Town.

Where it breaks: a shipping container is sealed and independent. Pods share a network identity with the cluster and are constantly created and destroyed — a container that is thrown away and rebuilt whenever convenient.

9. Best practices

Practice	Reason
Never run a bare Pod in production	Nothing recreates it. Use a Deployment, StatefulSet or Job.
One concern per container	You cannot scale, restart or resource-limit two concerns independently
Log JSON to stdout	The kubelet captures stdout. A log file inside a container is unreadable and vanishes on restart.
Run as non-root in the image and enforce it in the pod spec	Defence in depth: the image is correct, and the cluster verifies it
Pin image tags; never <code>:latest</code>	Otherwise what is running becomes unknowable and rollbacks stop working
Use the Downward API for pod identity	Enables per-pod diagnostics such as <code>/api/v1/_info</code>
Set <code>terminationGracePeriodSeconds</code> deliberately	Default 30s; payment workloads often need longer to drain

10. Common mistakes

Mistake	Symptom
Bare Pod in production	It dies and stays dead
Logging to a file inside the container	<code>kubectl logs</code> empty; logs lost on restart
Running as root	Container escape becomes host compromise
Expecting logs after a crash	container is waiting to start — use <code>--previous</code>
Using <code>:latest</code>	Nobody can say what is deployed
Two unrelated processes in one container	Cannot scale or restart them independently; no supervision
Assuming <code>Running</code> means healthy	Traffic sent to a pod that cannot serve

11. Security considerations

Threat	Control	Residual risk
Container escape via root	<code>runAsNonRoot</code> , <code>runAsUser: 10001</code> , drop all capabilities	Kernel vulnerabilities remain
Writable filesystem enables persistence	<code>readOnlyRootFilesystem: true</code> plus an <code>emptyDir</code> for scratch	Application must tolerate it
Privilege escalation via setuid	<code>allowPrivilegeEscalation: false</code>	—
hostPath mounts expose the node	Forbid via Pod Security Admission	Some system workloads legitimately need it
Secrets visible in <code>describe</code>	Use Secret volumes, not env vars; restrict RBAC	Secrets are base64, not encrypted (<i>see Day 3</i>)

12. Performance considerations

- **Startup time** is dominated by image pull. Keep images small; prefer multi-stage builds; pre-pull on nodes.
- **Init containers are serial** and add directly to startup latency. A slow `wait-for-db` delays every rollout.
- **Sidecars share the Pod's resource budget.** A log shipper with no limits can starve the application.
- **Pods per node** default to 110. On small nodes the practical limit is memory, not this number.

13. High availability

A single Pod has none. HA comes from running several via a controller, and from spreading them across nodes and zones with anti-affinity and topology spread constraints (*Day 4*). A Pod is bound to one node for its entire life — it is never migrated. "Rescheduling" means **deleting** it and **creating a new one** elsewhere.

14. Disaster recovery

Pods are disposable and hold no state worth recovering — unless they write to a volume, in which case the volume is what matters (*Day 3*). The recovery unit is the controller's manifest, not the Pod.

15. Monitoring

Metric	Threshold
<code>kube_pod_status_phase{phase="Pending"}</code>	Sustained > 2 min
<code>kube_pod_container_status_restarts_total</code> rate	Any sustained increase
<code>kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}</code>	Any
<code>kube_pod_status_ready</code>	0 while <code>Running</code>
<code>container_start_time_seconds</code>	Startup regression after a release

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>Pending</code>	No node fits; PVC unbound; taints	<code>kubectl describe pod</code> → Events	Add capacity, fix claim, tolerate taint
<code>ImagePullBackOff</code>	Bad tag, missing image, no credentials	<code>kubectl describe pod</code> → Events. Logs are useless.	Fix tag or build the image
<code>CrashLoopBackOff</code>	App exits repeatedly	<code>kubectl logs <pod> --previous</code>	Read the traceback
<code>OOMKilled</code> , exit 137	Exceeded memory limit	<code>kubectl describe pod</code> → Last State	Raise the limit or fix the leak
<code>Running</code> but <code>0/1</code>	Readiness probe failing	<code>kubectl describe pod</code> → Conditions	Check the dependency
Terminating forever	Finaliser or long grace period	<code>kubectl get pod -o json jq .metadata.finalizers</code>	Resolve the finaliser
<code>kubectl exec</code> fails	Multiple containers; no shell	<code>kubectl get pod -o jsonpath='{.spec.containers[*].name}'</code>	Use <code>-c <name></code> ; or <code>kubectl debug</code>

Interview questions

1. **Why does the Pod exist rather than scheduling containers directly?** *Because some containers must share a network namespace and filesystem to function — app plus sidecar, app plus init container. The Pod expresses "these must be co-located, share one IP, and live and die together".*

2. **What does the `pause` container do?** *It holds the Pod's network and IPC namespaces open so application containers can join them, and can restart independently without the Pod losing its IP.*
 3. **A pod is `Running` but shows `0/1`. What does that mean?** *The container process started but the readiness probe is failing, so the Pod has been removed from Service endpoints. It is alive but not receiving traffic — usually a dependency it needs is unavailable.*
 4. **`kubectl logs` returns "container is waiting to start". What now?** *(senior) There is no container yet, so there are no logs — this is an `ImagePullBackOff` or a volume-mount failure. Go to `kubectl describe pod` and read the Events, not the logs.*
 5. **How would you design a Pod for a service that must drain in-flight payments before terminating?** *(senior) A `preStop` hook that marks the pod unready and sleeps long enough for endpoint propagation, an application that handles SIGTERM by refusing new work while finishing in-flight requests, and `terminationGracePeriodSeconds` set comfortably above the longest expected request. Readiness must fail before the process stops accepting connections, or traffic is severed mid-authorisation.*
-

1.5 Deployments and ReplicaSets

1. What it is

A Deployment is a controller that keeps a stated number of identical Pods running, and manages the transition when you change them.

2. Why it exists

Bare Pods do not recover. Deployments add self-healing, scaling, and — crucially — a safe way to change what is running without downtime.

3. The business problem

The 02:14 page from §1.1. Also: AxisPay must deploy a new fraud model on a Tuesday afternoon, during trading, without dropping a single authorisation.

4. How it works

Deployment —owns→ ReplicaSet —owns→ Pods

A Deployment never creates a Pod. It creates and manages ReplicaSets; a ReplicaSet creates Pods.

This is not trivia. It is the mechanism behind every rolling update: the Deployment creates a **new** ReplicaSet and gradually shifts replicas from the old one to the new. Both continue to exist. **A rollback is simply scaling the old ReplicaSet back up** — no image pull, no new object, near-instant.

5. Internal architecture

Field	Meaning
<code>spec.replicas</code>	Desired count
<code>spec.selector.matchLabels</code>	Which pods this Deployment owns. Immutable.
<code>spec.template</code>	The Pod template. Changing it triggers a new ReplicaSet.
<code>spec.strategy</code>	<code>RollingUpdate</code> (default) or <code>Recreate</code>
<code>spec.revisionHistoryLimit</code>	Old ReplicaSets kept for rollback (default 10)

The ReplicaSet name is `<deployment>-<pod-template-hash>`. The hash is computed from `spec.template`, which is why changing the image creates a new ReplicaSet but changing `spec.replicas` does not.

6. Component interactions

```
you          kubectrl apply (new image)
Deployment ctrl computes a new pod-template-hash → creates ReplicaSet B
Deployment ctrl scales B up by maxSurge
ReplicaSet B  creates pods
kubelet       starts containers; readiness probes begin passing
Deployment ctrl sees new pods Ready → scales ReplicaSet A down
              repeats until B = desired and A = 0
              ReplicaSet A is RETAINED at 0 replicas, for rollback
```

7. Enterprise example

A payments platform deploys 40 times a day with `maxUnavailable: 0` and `maxSurge: 1`, so capacity never drops below 100% during a release. Every Deployment has a `PodDisruptionBudget` and anti-affinity across zones. A failed readiness probe halts the rollout automatically, leaving the previous version serving — the release fails safe, without a human deciding anything.

8. Real-world analogy

A shop manager told "there must always be three staff on the floor". Someone calls in sick; the manager calls in a replacement. That is the ReplicaSet.

Now the manager is told "swap all three for the new uniform, without the shop ever being understaffed" — bring in one new-uniform staff member, send one old-uniform member home, repeat. That is the Deployment.

Where it breaks: the manager reacts to events. The controller continuously compares the roster with the floor, regardless of what happened.

9. Best practices

Practice	Reason
Put only never-changing labels in <code>spec.selector</code>	It is immutable; fixing it means deleting and recreating the Deployment
Set <code>maxUnavailable: 0</code> for user-facing services	Capacity never drops during a release
Always define readiness probes	Without them a rollout "succeeds" while dropping traffic (<i>Day 2</i>)
Set <code>revisionHistoryLimit</code> explicitly	Default 10 ReplicaSets per Deployment adds up across a platform
Record a change cause	<code>kubectl rollout history</code> becomes readable during an incident
Use <code>kubectl rollout status</code> in CI	Makes the pipeline wait for, and fail on, a bad release
Prefer <code>Recreate</code> only for singletons that cannot run two copies	E.g. a schema migration job

10. Common mistakes

Mistake	Symptom
Version number in <code>spec.selector</code>	Every release orphans the previous pods
Selector not a subset of template labels	Rejected at admission — the message is not obvious
<code>kubectl scale</code> without updating YAML	Silent revert on next apply
No readiness probe	Rollout completes; traffic is dropped
Relabelling a running pod	It leaves both its Service and its ReplicaSet; a replacement appears; you now have an orphan
Expecting the Deployment to own Pods directly	Cannot explain rollback; confused by ReplicaSets in <code>get all</code>
Deleting a ReplicaSet by hand	The Deployment recreates it immediately

11. Security considerations

Threat	Control	Residual risk
Anyone with <code>update</code> on Deployments can run arbitrary images	RBAC per namespace; admission control on registries	CI credentials remain powerful
A rollback re-introduces a vulnerable image	Scan on pull; block known-bad digests at admission	Rollback during an incident may bypass policy
Deployment mounting a Secret it should not	RBAC on Secrets; admission policy	Namespace admins can read their Secrets

12. Performance considerations

- **Rollout speed** is bounded by `maxSurge`, readiness-probe timing and image pull. `initialDelaySeconds` set too high makes every release slow.
- `maxUnavailable: 0` **requires spare capacity** — the cluster must fit `replicas + maxSurge` pods during the rollout.
- **Large replica counts** amplify everything: 200 replicas at `maxSurge: 1` takes a very long time. Use a percentage.
- **Old ReplicaSets** consume etcd space and API server memory. Keep `revisionHistoryLimit` modest.

13. High availability

Replicas alone are not HA — three replicas on one node all die with that node. Real availability needs:

- Replicas ≥ 3 across ≥ 3 nodes
- Pod anti-affinity or topology spread constraints (*Day 4*)
- PodDisruptionBudget so voluntary disruptions (drains, upgrades) cannot take them all (*Day 4*)
- Readiness probes so traffic only reaches pods that can serve (*Day 2*)

14. Disaster recovery

The Deployment manifest **is** the recovery artefact. Keep it in Git and a lost Deployment is one `kubectl apply` away. Rollback within the cluster is `kubectl rollout undo`, limited by `revisionHistoryLimit`. Beyond that horizon, recovery means re-applying an older manifest from Git — another reason the repository, not the cluster, should be the source of truth.

15. Monitoring

Metric	Threshold
<code>kube_deployment_status_replicas_available</code> vs <code>kube_deployment_spec_replicas</code>	Gap > 5 min — the single most useful deployment alert
<code>kube_deployment_status_observed_generation</code> vs <code>metadata.generation</code>	Lagging = controller not reconciling
<code>kube_replicaset_status_replicas</code> per RS	Two RSs both non-zero for a long time = stuck rollout
Restart rate across the Deployment	Rising after a release = bad build

The alert that would have caught Day 1's incident before a merchant phoned: `promql`
`kube_deployment_status_replicas_available{deployment="payment-service"} <`
`kube_deployment_spec_replicas{deployment="payment-service"}` for 5 minutes. You will
 deploy exactly this on Day 5.

16. Troubleshooting

Symptom	Cause	Command	Fix
Rollout stuck, never completes	New pods never Ready	<code>kubectl rollout status ;</code> <code>kubectl describe pod</code>	Fix the probe or the app
ProgressDeadlineExceeded	Rollout exceeded <code>progressDeadlineSeconds</code> (default 600)	<code>kubectl describe deploy</code>	Investigate the new pods
Deleted pod not replaced	You deleted the Deployment, not a Pod	<code>kubectl get deploy</code>	Re-apply
Replica count keeps changing on its own	HPA active, or overlapping selectors	<code>kubectl get hpa ; compare selectors</code>	Expected, or make selectors disjoint
Rollback does nothing	History pruned by <code>revisionHistoryLimit</code>	<code>kubectl rollout history</code>	Re-apply an older manifest from Git
More pods than <code>replicas</code>	An orphaned pod from a relabel	<code>kubectl get pods --show-labels</code>	Delete the orphan

Interview questions

1. **What owns a Pod created by a Deployment?** *The ReplicaSet. The Deployment owns the ReplicaSet; the ReplicaSet owns the Pods. This is the mechanism behind rolling updates and rollbacks.*
 2. **Why does Kubernetes keep old ReplicaSets at zero replicas?** *So a rollback is just scaling one back up — no image pull, no new object, near-instant. `revisionHistoryLimit` controls how many are kept.*
 3. **Why is `spec.selector` immutable?** *Because changing which pods a controller owns mid-flight would orphan or double-own running pods. Kubernetes forbids it; fixing a selector requires deleting and recreating the Deployment — in production, with traffic on it.*
 4. **You set `maxUnavailable: 0` and `maxSurge: 1` with 2 replicas. Walk through the rollout.** *(senior) One new pod is created (3 total). When it becomes Ready, one old pod is terminated (2 total). Repeat. Capacity never drops below 2. It requires headroom for one extra pod and it is slower than a permissive strategy — the correct trade for a payment path.*
 5. **A rollout is stuck at 2/3 updated. How do you diagnose it?** *(senior) `kubectl rollout status` for the stall, then `kubectl get pods` to find the pod that is not Ready, then `describe` for probe failures and events, then `logs` for the application's own account. Most often it is a readiness probe failing against an unavailable dependency, or a resource limit that the new version exceeds.*
-

1.6 Services

1. What it is

A Service is a stable virtual IP and DNS name that load-balances to a continuously updated set of Pods, selected by label.

2. Why it exists

Self-healing makes Pods disposable — and their IP addresses go with them. Something must hold a stable address on their behalf.

3. The business problem

A junior engineer hard-coded a pod IP into the gateway configuration. It worked for four hours. Then the pod was rescheduled, got a new IP, and every card authorisation failed until someone noticed.

4. How it works

```
Service (selector) → EndpointSlice → kube-proxy → kernel rules → Pods
```

1. You define a Service with a **label selector**.
2. The **endpoint controller** continuously evaluates that selector and writes matching, *ready* pod IPs into an **EndpointSlice**.
3. **kube-proxy** on every node watches EndpointSlices and programs iptables or IPVS rules.
4. Traffic to the ClusterIP is rewritten by the **kernel** to one of the pod IPs.

The direction of causation is what people get wrong. A Service does not *contain* pods. It *selects* them, continuously. Change a pod's labels and it silently leaves the Service. Nothing errors.

There is **no proxy process in the request path** — it is kernel rules. That is why a ClusterIP Service adds almost no latency.

5. Internal architecture

Type	Behaviour	Use
<code>ClusterIP</code>	Virtual IP reachable only inside the cluster (default)	Service-to-service — all of Day 1
<code>NodePort</code>	Opens the same port on every node	Development; behind an external LB
<code>LoadBalancer</code>	Provisions a cloud load balancer	Production external exposure
<code>ExternalName</code>	Returns a CNAME; no proxying	Aliasing an external dependency
Headless (<code>clusterIP: None</code>)	No virtual IP; DNS returns pod IPs	StatefulSets, client-side balancing (<i>Day 3</i>)

DNS forms (mechanism covered on Day 4):

Form	Resolves from
<code>payment-service</code>	Same namespace only
<code>payment-service.axispay-core</code>	Anywhere
<code>payment-service.axispay-core.svc.cluster.local</code>	Anywhere — fully qualified

6. Component interactions

```

you          create Service with selector
endpoint ctrl lists pods matching the selector AND Ready
              writes their IPs into an EndpointSlice
kube-proxy (all) watches EndpointSlices → programs iptables/IPVS
CoreDNS       watches Services → answers <svc>.<ns>.svc.cluster.local
client pod    resolves the name → gets ClusterIP
              connects → kernel DNATs to a pod IP

```

Only `Ready` pods are included. This is the direct link between readiness probes and traffic routing, and it is why a readiness probe is the mechanism behind zero-downtime deployments.

7. Enterprise example

A payments platform exposes every internal service as ClusterIP and admits external traffic only through an Ingress in a DMZ namespace. Internal calls use fully qualified DNS names supplied by ConfigMap, so the same manifests promote unchanged from dev to production. Session affinity is deliberately **off** — payment requests are stateless and must be free to land anywhere.

8. Real-world analogy

A company switchboard number. Staff come and go and change desks; the switchboard number never changes and always connects you to whoever is currently on duty.

Where it breaks: a switchboard operator is a process in the middle. A Kubernetes Service is kernel rules — after the connection is established there is nothing in the path at all.

9. Best practices

Practice	Reason
Use named <code>targetPort</code>	The container can change its port without the Service changing
Keep selectors minimal and stable	Fewer labels, fewer ways to break silently
ClusterIP by default; expose only at the edge	Every NodePort is an attack surface
Use FQDNs in configuration	Removes an entire class of cross-namespace bug
Check EndpointSlices, not just the Service	A Service always has an IP; that proves nothing
Avoid session affinity unless genuinely required	It defeats load balancing and complicates rollouts
Remember load balancing is per connection	HTTP keep-alive and gRPC pin to one pod

10. Common mistakes

Mistake	Symptom
Selector does not match pod labels	Service exists, has an IP, has no endpoints . Nothing errors.
<code>targetPort</code> wrong	Endpoints exist; connections refused
Short DNS name across namespaces	Name resolution failure
Assuming per-request load balancing	One pod gets all traffic over a keep-alive connection
Expecting <code>kubectl get svc</code> to reveal the problem	It never shows endpoint health
Exposing internal services with NodePort	Unnecessary external exposure

11. Security considerations

Threat	Control	Residual risk
Any pod can reach any Service by default	NetworkPolicy default-deny (<i>Day 4</i>)	Requires a policy-enforcing CNI
NodePort exposes a service on every node	Prefer Ingress; restrict with firewall rules	Node IPs may be reachable internally
Service name enumeration reveals architecture	RBAC on <code>list services</code> ; NetworkPolicy	DNS is broadly readable in-cluster
Traffic between pods is plaintext	mTLS via a service mesh (beyond this course)	Meshes add significant operational cost

12. Performance considerations

- **iptables mode** is $O(n)$ in rule evaluation; it degrades noticeably beyond a few thousand Services. **IPVS mode** uses hash tables and scales much better.
- **EndpointSlices** (replacing the older Endpoints object) exist precisely to avoid shipping one enormous object to every node on every pod change.
- **DNS is often the real latency cost.** The default `ndots:5` means short names generate several failed lookups before the right one. Using FQDNs avoids it (*Day 4*).
- `externalTrafficPolicy: Local` preserves the client source IP and avoids an extra hop, at the cost of uneven balancing.

13. High availability

Services are highly available by construction: the ClusterIP is a kernel rule present on **every** node, so there is no single point of failure and nothing to fail over. If kube-proxy dies on a node, existing connections survive; only rule *updates* stop.

Availability depends on having ready endpoints — which depends on replicas, spread and readiness probes.

14. Disaster recovery

Services are pure configuration and trivially reproducible from manifests. The one caveat: a `LoadBalancer` Service usually holds an allocated external IP. Recreating it may allocate a **different** IP, which breaks DNS and any allowlists your merchants maintain. In production, reserve static IPs and reference them explicitly.

15. Monitoring

Metric	Threshold
<code>kube_endpoint_address_available</code>	0 while the Service exists — page
<code>kube_endpoint_address_not_ready</code>	Sustained > 0
Endpoint count vs replica count	Persistent mismatch
<code>kubeproxy_sync_proxy_rules_duration_seconds</code>	Rising = too many Services/endpoints
CoreDNS request/error rate	Errors indicate DNS problems before services notice

16. Troubleshooting

Symptom	Cause	Command	Fix
No endpoints	Selector mismatch, or no pod is Ready	<code>kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc></code>	Compare <code>svc.spec.selector</code> with <code>pod.metadata.labels</code>
Endpoints exist, connection refused	Wrong <code>targetPort</code>	<code>kubectl get svc <s> -o yaml</code>	Match the container port
Works in-namespace, fails across	Short name used	<code>nslookup</code> from inside a pod	Use the FQDN
One pod gets all traffic	Client keep-alive	Test with separate connections	Expected — not a Service bug
Intermittent failures	Some pods not Ready	<code>kubectl get pods -o wide</code>	Fix readiness
Nothing resolves at all	CoreDNS unhealthy	<code>kubectl -n kube-system get pods -l k8s-app=kube-dns</code>	Restart CoreDNS

Interview questions

- 1. A Service has a ClusterIP but requests fail. What is the first thing you check?** *The EndpointSlice. A Service always has an IP whether or not its selector matches anything, so `kubectl get svc` looks healthy either way. "No endpoints" is the single most common Service bug.*
- 2. How does a Service know which pods to send traffic to?** *It does not "know" — it selects. The endpoint controller continuously evaluates the label selector and writes matching, ready pod IPs into an EndpointSlice. kube-proxy programs the kernel from that.*

3. **Is there a proxy process in the request path?** *No. kube-proxy programs iptables or IPVS rules; the kernel does the rewriting. That is why ClusterIP adds almost no latency.*
 4. **Why does gRPC need special handling behind a Kubernetes Service?** *(senior) Because kube-proxy load-balances connections, not requests. gRPC multiplexes many requests over one long-lived HTTP/2 connection, so all of them pin to a single pod. Solutions are client-side load balancing with a headless Service, a service mesh, or a proxy that understands HTTP/2.*
 5. **You relabel a running pod. What happens to the Service and the ReplicaSet?** *(senior) It leaves both. The endpoint controller drops it from the EndpointSlice, so it stops receiving traffic. The ReplicaSet no longer counts it, observes a shortfall, and creates a replacement. You end up with an orphaned pod that nothing manages and that will survive deleting the Deployment.*
-

1.7 Java Application Lifecycle in Kubernetes

1. What it is

This is the lifecycle of a Java service **inside** a Kubernetes Pod: image starts, the JVM boots, Spring Boot builds its application context, probes decide whether the Pod may receive traffic, and later the kubelet asks it to terminate.

For a Spring Boot service on JDK 17, that lifecycle is rarely instant. Class loading, bean creation, database pool initialisation and cache warm-up make Java behave very differently from a tiny Go binary that starts in 200ms.

2. Why it exists

Kubernetes only sees a container process. It does **not** understand the internal states of a JVM application unless you expose them through probes and shutdown behaviour.

That gap matters because the platform makes traffic and restart decisions long before a human is looking. If Kubernetes thinks a process is healthy too early, traffic arrives before Spring has finished starting. If Kubernetes kills a Pod too fast, in-flight requests die halfway through card authorisation.

3. The business problem

AxisPay learned this the hard way during a Thursday afternoon rolling update of `payment-service`. The old Pods were serving long-running card authorisations to a downstream issuer simulator. Each request held a database connection from HikariCP while the authorisation result and ledger reservation were being coordinated.

The Deployment used the default `terminationGracePeriodSeconds: 30`. The application had no graceful shutdown enabled, and the `preStop` hook slept for 10 seconds in the hope that traffic would drain. It did not. Endpoint removal, keep-alive clients and JVM shutdown raced each other. At 30 seconds the kubelet sent `SIGKILL`. Two Pods died with requests still active, four authorisations were retried by merchants, and one merchant saw a temporary double-hold on a customer card.

The lesson was not "Java is slow". The lesson was that Kubernetes termination timing must be designed around **application reality**, not defaults.

4. How it works

Startup and shutdown are separate problems.

Startup

```
kubelet starts container
JVM starts PID 1
Spring Boot loads classes and creates beans
connection pools initialise
embedded Tomcat/Netty binds ports
startupProbe succeeds
readinessProbe succeeds
Service starts sending traffic
```

For Spring Boot applications, a `startupProbe` is usually the missing control. Many AxisPay services take 30–90 seconds to start after a cold node boot because they load hundreds of classes, build the Spring context, connect to PostgreSQL and register Actuator health indicators.

Without a `startupProbe`, a liveness probe starts judging the container too early. Kubernetes then kills a perfectly normal slow-starting JVM and turns startup delay into

`CrashLoopBackOff`.

Shutdown

```
pod marked Terminating
terminationGracePeriodSeconds countdown starts
kubelet runs preStop hook
kubelet sends SIGTERM to PID 1
JVM begins shutdown sequence
registered shutdown hooks run
Spring Boot stops accepting new work and drains existing work
when grace expires, kubelet sends SIGKILL
```

A critical nuance: the JVM does not magically know how to drain HTTP requests or finish business transactions. `SIGTERM` starts JVM shutdown, but **graceful application shutdown exists only if the app registers shutdown hooks and cooperates**.

Spring Boot gives you that cooperation when configured correctly:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=60s
```

The first tells the embedded server to stop accepting new requests and wait for active ones to finish. The second sets how long Spring should wait for beans to stop cleanly. If your Pod's `terminationGracePeriodSeconds` is 30 but Spring is configured to wait 60, Kubernetes wins. The process is killed at 30.

5. Internal architecture

Layer	Responsibility	Failure mode if misconfigured
kubelet	Starts containers, runs probes, sends termination signals	Kills a slow-starting or slow-stopping app too early
startupProbe	Says "do not evaluate liveness yet"	Normal boot looks like a crash
readinessProbe	Says whether the Pod may receive traffic	Traffic sent before app is really usable
livenessProbe	Says whether the process should be restarted	Probe loop turns transient slowness into restarts
JVM	Runs the process and shutdown hooks	No business-level draining by default
Spring Boot	Builds the app context and coordinates graceful stop	Long startup, partial shutdown, hanging beans
Connection pools / worker pools	Hold in-flight work during shutdown	Transactions terminated mid-flight

6. Component interactions

A safe startup for `payment-service` looks like this:

```

startupProbe:
  httpGet:
    path: /startupz
    port: 8080
  failureThreshold: 30
  periodSeconds: 3
readinessProbe:
  httpGet:
    path: /readyz
    port: 8080
  periodSeconds: 5
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  periodSeconds: 10
terminationGracePeriodSeconds: 90

```

That gives Spring Boot up to 90 seconds to initialise before liveness is allowed to act, and up to 90 seconds to drain at shutdown.

The shutdown race is the harder part:

1. Pod is removed from the Service endpoint set only after readiness fails and controllers propagate the change.
2. Existing clients may keep connections open for several seconds.

3. `preStop` consumes time **inside** `terminationGracePeriodSeconds`.
4. `SIGTERM` reaches the JVM only after the hook phase begins.
5. If the grace period expires first, `SIGKILL` ends the discussion.

This is why "sleep 30" in `preStop` is not a design. It is wishful thinking.

7. Enterprise example

In production, AxisPay's `auth-service`, `payment-service` and `core-service` all expose three Actuator-backed endpoints: `/startupz`, `/readyz` and `/healthz`. Their Deployments deliberately set a longer grace period than the application shutdown timeout.

Service	Cold start	Shutdown timeout	Pod grace
<code>auth-service</code>	20–35s	20s	45s
<code>payment-service</code>	45–75s	60s	90s
<code>core-service</code>	60–90s	75s	120s

The numbers are boring by design. They come from measurement, not folklore.

8. Real-world analogy

A restaurant opening and closing. Turning the lights on does not mean the kitchen is ready; chefs still need prep time. At closing time, locking the front door must happen **before** you throw out the customers already eating.

Where it breaks: restaurants do not have an external control loop that will forcibly bulldoze the building after 30 seconds if the staff are still cleaning up.

9. Best practices

Practice	Reason
Add a <code>startupProbe</code> to every non-trivial Spring Boot service	Prevents liveness from killing slow but healthy startups
Keep readiness and liveness semantically different	Readiness answers "can serve?"; liveness answers "should restart?"
Enable Spring graceful shutdown	Gives the JVM an application-level drain path
Set <code>terminationGracePeriodSeconds</code> above the longest legitimate request	Default 30s is often too short for payment flows
Make <code>preStop</code> fail readiness first, then wait briefly	Stops new traffic before <code>SIGTERM</code> drainage begins
Measure startup and drain times per service	Java services differ materially; copy-paste values are dangerous

10. Common mistakes

Mistake	Symptom
No <code>startupProbe</code> on a 60-second Spring Boot app	<code>CrashLoopBackOff</code> during node restarts or new rollouts
Same endpoint for readiness and liveness	Healthy-but-dependent apps get restarted instead of drained
<code>terminationGracePeriodSeconds</code> left at default 30	In-flight payments cut off during rolling updates
Long <code>preStop</code> sleep with no readiness change	Pod keeps receiving traffic while supposedly draining
Assuming <code>SIGTERM</code> is enough	Process exits, but business work is not finished safely

11. Security considerations

Threat	Control	Residual risk
Terminating pods still accept traffic and expose stale auth decisions	Fail readiness first and drain quickly	Existing keep-alive sessions may persist briefly
Probes expose internal state too broadly	Expose only minimal health endpoints; protect richer Actuator endpoints	Internal callers may still fingerprint service behaviour
Shutdown hooks log sensitive state during failure	Review exception logging paths	Emergency diagnostics still risk oversharing

12. Performance considerations

- JVM startup time is workload-specific. Classpath size, reflection-heavy frameworks, TLS initialisation and connection pool warm-up all matter.
- Aggressive liveness probes create self-inflicted load during startup by opening many failing connections.
- Graceful shutdown increases rollout duration because old Pods remain alive longer; plan capacity accordingly.
- A service that drains for 90 seconds but runs with `maxUnavailable: 1` needs enough headroom for both old and new replicas during deployment.

13. High availability

High availability is not just replica count. A three-replica Deployment with bad lifecycle settings can take all three replicas out one by one during a rollout.

Safe HA for Java services needs:

- multiple replicas
- readiness probes that remove broken Pods quickly
- startup probes that protect slow boots
- graceful shutdown so rolling updates do not sever requests
- surge capacity during updates

14. Disaster recovery

Lifecycle settings are configuration, so recovery is straightforward **if** they live in Git. The DR risk is not losing the settings; it is forgetting them during a rebuild and reintroducing a latent outage pattern.

For regulated services such as `core-service`, lifecycle values should be part of the platform baseline and peer-reviewed like any other control.

15. Monitoring

Signal	Why	Alert at
Pod startup duration	Detects regression after a release	> historical p95 by 50%
<code>kube_pod_container_status_restarts_total</code>	Startup probe/liveness misdesign shows here first	Sustained increase
Termination duration from app logs	Proves whether drain fits grace	> 80% of grace budget
Readiness flaps during rollout	Pods becoming ready too early or too late	Any sustained flapping
HTTP 5xx during Deployment updates	The user-visible symptom of bad shutdown	Spike during rollout

16. Troubleshooting

Symptom	Likely cause	Command	Fix
<code>CrashLoopBackOff</code> only on fresh nodes	JVM startup exceeds liveness budget	<code>kubectl describe pod ; kubectl logs --previous</code>	Add or relax <code>startupProbe</code>
Pod stuck <code>Running</code> 0/1 for 2 minutes then becomes healthy	Normal slow start, readiness tuned too tightly	<code>kubectl describe pod → probe events</code>	Increase readiness thresholds; add <code>startupProbe</code>
Rolling update causes 502s	Pod exits before requests drain	<code>kubectl describe pod ; app shutdown logs</code>	Enable graceful shutdown; raise grace period
Pod sits <code>Terminating</code> until force-killed	Spring timeout longer than Pod grace	Compare app config vs manifest	Align <code>spring.lifecycle...</code> with <code>terminationGracePeriodSeconds</code>
<code>preStop</code> runs but traffic still arrives	Readiness never failed or endpoint propagation lag	<code>kubectl get endpointslice -w</code>	Fail readiness first, then wait briefly

Interview questions

- Why do Spring Boot services often need a `startupProbe` when a Node.js or Go service might not?** Because the JVM and Spring context can take tens of seconds to initialise. Without a `startupProbe`, Kubernetes begins liveness checks during normal boot and mistakes slow startup for failure.
- What does `server.shutdown=graceful` actually change?** It tells Spring Boot's embedded server to stop accepting new requests and wait for active ones to complete during shutdown, instead of terminating abruptly. It is application-level cooperation layered on top of `SIGTERM`.
- Why is `terminationGracePeriodSeconds` not just a nice-to-have for payment workloads?** Because card authorisations and ledger writes are real in-flight financial operations. If the kubelet sends `SIGKILL` before they drain, you create retries, duplicate holds, inconsistent state and merchant-visible errors.
- Explain the relationship between `preStop`, `SIGTERM` and the grace period.** (senior) The grace-period timer starts first. `preStop` runs inside that budget. `SIGTERM` and shutdown hooks then have only the remaining time. If the whole sequence exceeds the budget, the kubelet sends `SIGKILL`. Therefore `preStop` must be short and purposeful, not a blind sleep.

5. How would you choose probe timings for a Java service? *(senior) Measure real cold-start and warm-start times, then set `startupProbe` to cover the worst legitimate startup. Make readiness reflect business readiness, not just port-open. Make liveness conservative and aimed at unrecoverable stuck states, not dependency blips.*

1.8 Init Containers and Dependency Management

1. What it is

An init container is a container that runs **before** the main application containers in a Pod. It must complete successfully before the next init container, and eventually the main application, can start.

Init containers exist to do setup work that should not stay running for the whole life of the Pod.

2. Why it exists

Java microservices usually assume the world already exists: the database is reachable, schemas are current, trust stores are present, and configuration files are mounted. In Kubernetes that assumption is often false, especially during a fresh cluster build or after a dependency restart.

If the main Spring Boot container is given those responsibilities directly, it becomes harder to reason about failure. You see a `CrashLoopBackOff`, but you do not know whether the app code is broken or merely waiting for PostgreSQL.

Init containers separate preparation from serving.

3. The business problem

AxisPay's `core-service` owns the ledger tables used for reservations, postings and reversals. On a new environment build, the service must not start serving traffic until the schema exists and is the expected version.

A team once embedded Flyway migration logic inside the main application startup and also wrapped the container entrypoint with a home-grown `wait-for-it.sh`. When PostgreSQL was slow to recover, the shell loop masked the real problem for 14 minutes. When Flyway later failed on a locking error, the container crashed without a clean audit trail. Operations saw only a Java stack trace and assumed the application release was bad.

The fix was to move dependency waiting and schema migration into explicit init containers, where Kubernetes can show their state directly.

4. How it works

Init containers run in order:

```
init container 1  -> must exit 0
init container 2  -> must exit 0
application      -> may now start
```

If any init container fails, the Pod does not move on. Kubernetes retries according to the Pod restart policy, and `kubectl describe pod` shows exactly which init container is blocked.

A realistic AxisPay example for `core-service` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: core-service
  namespace: axispay-core
spec:
  replicas: 3
  selector:
    matchLabels:
      app: core-service
  template:
    metadata:
      labels:
        app: core-service
    spec:
      initContainers:
        - name: wait-for-postgres
          image: postgres:16
          command:
            - sh
            - -c
            - |
              until pg_isready -h ledger-postgres.axispay-data.svc.cluster.local -p 5432 -U
"$DB_USER"; do
              echo "waiting for ledger database"
              sleep 2
            done
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: core-db-credentials
                  key: username
            - name: migrate-ledger-schema
              image: flyway/flyway:10.17.0
              args:
                - -url=jdbc:postgresql://ledger-postgres.axispay-data.svc.cluster.local:5432/
axispay_core
                - -user=$(DB_USER)
                - -password=$(DB_PASSWORD)
                - -connectRetries=30
                - migrate
              env:
                - name: DB_USER
                  valueFrom:
                    secretKeyRef:
                      name: core-db-credentials
                      key: username
                - name: DB_PASSWORD
                  valueFrom:
                    secretKeyRef:
                      name: core-db-credentials
                      key: password
              volumeMounts:
                - name: ledger-migrations
                  mountPath: /flyway/sql

```



```

containers:
  - name: core-service
    image: registry.axispay.internal/core-service:1.0.0
    ports:
      - containerPort: 8080
volumes:
  - name: ledger-migrations
    configMap:
      name: core-ledger-migrations

```

The main application starts only after PostgreSQL answers and Flyway exits 0.

5. Internal architecture

Element	Responsibility	Operational value
init container image	Holds tools not needed in the app image	Keeps app image smaller and cleaner
sequential ordering	Enforces dependency order	Prevents race conditions during startup
exit code	Tells Kubernetes success or failure	Makes diagnosis explicit
shared Pod network namespace	Lets init containers reach cluster Services normally	No special networking required
shared volumes	Pass files or scripts to the main container	Clean hand-off

6. Component interactions

```

kubelet      starts Pod sandbox
kubelet      runs init container 1: wait-for-postgres
PostgreSQL   accepts connections
init 1       exits 0
kubelet      runs init container 2: Flyway migrate
Flyway       creates/updates ledger tables
init 2       exits 0
kubelet      starts main Spring Boot container
readinessProbe eventually passes
Service      adds pod as endpoint

```

The important guarantee is ordering. Kubernetes will never run `migrate-ledger-schema` before `wait-for-postgres` succeeds, and will never start `core-service` while a migration is still running.

7. Enterprise example

AxisPay uses init containers in three repeatable patterns:

Service	Init responsibility	Why
<code>core-service</code>	Flyway schema migration	Ledger tables must exist before posting money
<code>payment-service</code>	Fetch merchant routing rules from internal config store	Faster main-container startup; clearer failure scope
<code>merchant-service</code>	Wait for RabbitMQ and pre-create webhook dead-letter queues	Prevents consumer startup against partial messaging state

This pattern also makes audits easier. A failed schema migration is visible as an infrastructure-prep failure, not buried in generic application logs.

8. Real-world analogy

A theatre opening for a performance. The audience cannot enter until the cleaners finish, the stage lights are tested and the ticket scanners are online. Those setup tasks happen first, in order, and none of them should still be "running forever" once the show begins.

Where it breaks: in a theatre, setup staff can improvise around a failure. Init containers cannot; any failure stops progress completely.

9. Best practices

Practice	Reason
Keep init containers single-purpose	Easier to debug and safer to rerun
Use purpose-built images such as <code>postgres</code> or <code>flyway</code>	Avoid stuffing admin tools into the app image
Fail fast with clear log output	Operators should know <i>which</i> dependency is missing
Put schema migration ownership with the service that owns the schema	Prevents hidden cross-service coupling
Prefer init containers over shell wrappers in <code>command:</code>	Kubernetes can observe init state directly
Set resource requests on init containers too	Large migrations can starve small nodes

10. Common mistakes

Mistake	Symptom
Using <code>wait-for-it</code> inside the main container entrypoint	App logs are noisy and failure scope is unclear
Putting infinite wait loops in init containers	Pods appear hung forever with no escalation path
Reusing the application image for migrations	Bloated image, wider attack surface, slower pulls
Running destructive migrations automatically on every replica	Lock contention or duplicate DDL attempts
Forgetting Secret or ConfigMap mounts for init containers	Main app never starts, but the reason is in init failures

11. Security considerations

Threat	Control	Residual risk
Migration container has powerful DB credentials	Use a dedicated migration role with only needed DDL/DML rights	Migration role is still sensitive and must be rotated
Init image from public registry is compromised	Pin image digest and scan images	Supply-chain risk remains
Secrets exposed to both init and app unnecessarily	Scope env vars and mounts only to containers that need them	Pod-level compromise still exposes mounted secrets

12. Performance considerations

- Init containers are serial, so the total startup time is the sum of all init work plus app boot time.
- Database migration that takes 45 seconds on one replica takes 45 seconds on **every** new Pod unless you design ownership carefully.
- Large tool images increase pull time during node replacement; keep them lean and cached.
- A hanging dependency check can block an entire rollout even when the application build is correct.

13. High availability

Init containers improve availability indirectly by ensuring the main container starts in a known-good state. However, they also create a dependency chain: if PostgreSQL is down, new replicas cannot come up.

That means HA planning must include the dependencies, not just the app Deployment. If `core-service` needs PostgreSQL to start, a database outage reduces your ability to self-heal or scale out.

14. Disaster recovery

In a DR rebuild, init containers are often what makes first startup succeed. The database restore must happen before services can migrate against it, and migration scripts themselves are part of the recovery artefact.

For financial workloads, store migration definitions in Git and treat them as controlled changes. A restored database with missing or out-of-order migrations is worse than a service that fails loudly.

15. Monitoring

Signal	Why
Pod condition <code>Initialized=False</code>	Immediately points to init-stage failures
Time spent in <code>Init: states</code>	Detects slow dependency readiness or slow migrations
Exit code / restart count of init containers	Shows flapping setup logic
Database migration duration	Predicts rollout time and lock risk

16. Troubleshooting

Symptom	Likely cause	Command	Fix
<code>Init:ImagePullBackOff</code>	Init image tag wrong or registry auth missing	<code>kubectl describe pod</code>	Fix init image reference or pull secret
<code>Init:CrashLoopBackOff</code> after Flyway run	Migration script exits non-zero	<code>kubectl logs <pod> -c migrate-ledger-schema</code>	Fix SQL or DB permissions
Pod stuck at <code>Init:0/2</code>	First init is hanging on an unavailable dependency	<code>kubectl logs <pod> -c wait-for-postgres</code>	Fix the dependency; add timeout and clearer logging
Main app never starts though image is fine	Earlier init container failed	<code>kubectl get pod -o yaml</code> → <code>initContainerStatuses</code>	Repair the init step, not the app
Rollout much slower than expected	Heavy init work on every Pod	Measure <code>Init:</code> duration	Refactor one-off prep out of per-Pod startup

Interview questions

1. **What guarantee do init containers give you that a shell script in the app entrypoint does not?** *Kubernetes understands their lifecycle explicitly. It reports which init step failed, preserves logs per init container, and guarantees sequential execution before the app container starts.*
 2. **Why are init containers a better place for Flyway or Liquibase than the main Spring Boot process?** *Because schema preparation becomes a visible, isolated startup phase. It removes migration failure from normal application crash diagnosis and lets the app image stay focused on serving traffic.*
 3. **What is the `wait-for-it` anti-pattern?** *Putting indefinite dependency waiting inside the main container startup wrapper. It hides the true state from Kubernetes, delays diagnosis and often leads to fragile shell logic instead of observable init steps.*
 4. **What happens if the second init container fails but the first succeeded?** *(senior) The Pod stays in init failure and Kubernetes retries according to restart policy. The first init step is considered complete, but the Pod never progresses to app containers until the failing second step eventually exits successfully.*
 5. **Why can init containers reduce, rather than increase, operational complexity even though they add more YAML?** *(senior) Because they turn ambiguous startup behaviour into explicit phases with separate images, logs and exit codes. More declarative structure usually means less guesswork during incidents.*
-

1.9 Sidecar Patterns for AxisPay Services

1. What it is

A sidecar pattern is when multiple containers run in the same Pod and one of them exists to support the main application rather than serve business traffic directly.

At AxisPay, a production `payment-service` Pod commonly runs three containers: the Spring Boot application, a log-shipping sidecar, and a metrics sidecar.

2. Why it exists

Some platform functions belong close to the application but should not be compiled into it: log shipping, metrics translation, service-mesh proxies, certificate renewal and local agents.

A sidecar keeps that concern deployable with the workload while remaining independently replaceable.

3. The business problem

`payment-service` writes structured JSON logs and exposes JVM internals through JMX. The platform team wants all logs shipped to the central store and all JVM metrics exposed to Prometheus without forcing every application team to embed vendor-specific libraries or maintain custom exporters.

If those features were baked into the Java service itself, every application release would become an observability release too. A logging config change would require rebuilding `payment-service`. That is the wrong ownership boundary.

4. How it works

All containers in a Pod share:

- one network namespace
- one IP address
- the loopback interface (`localhost`)
- any explicitly mounted shared volumes

That makes a sidecar powerful. A metrics sidecar can scrape the application on `localhost` without a Service. A logging sidecar can read files written to a shared volume. The main container keeps its simple contract: serve the application.

A typical production shape:

```
spec:
  volumes:
    - name: app-logs
      emptyDir: {}
  containers:
    - name: payment-service
      image: registry.axispay.internal/payment-service:1.0.0
      volumeMounts:
        - name: app-logs
          mountPath: /var/log/axispay
    - name: fluent-bit
      image: cr.fluentbit.io/fluent/fluent-bit:3.1.8
      volumeMounts:
        - name: app-logs
          mountPath: /var/log/axispay
    - name: jmx-exporter
      image: bitnami/jmx-exporter:1.0.1
      args:
        - --web.listen-address=:9404
        - --config.file=/config/jmx.yaml
```

The application writes JSON logs to `/var/log/axispay/application.log` ; Fluent Bit tails them; the JMX exporter exposes metrics for Prometheus.

5. Internal architecture

Container	Responsibility	Shared resource
<code>payment-service</code>	Business logic, HTTP API, database calls	CPU, memory, Pod IP, shared log volume
<code>fluent-bit</code> or <code>filebeat</code>	Tails and forwards logs	Shared log volume, network namespace
<code>jmx-exporter</code>	Converts JVM/JMX metrics to Prometheus format	<code>localhost</code> , config volume, Pod IP

Two details matter operationally:

1. Because all containers share `localhost` , the exporter can scrape `payment-service` on `localhost:9404` or its JMX port without a separate Service.
2. Because resources are accounted per container but enforced at the Pod boundary by node memory pressure, a noisy sidecar can contribute to a Pod OOM that developers misread as "the app crashed".

6. Component interactions

```
payment-service  writes JSON logs to shared volume
fluent-bit       tails file and ships to central logging
payment-service  exposes JMX / local metrics endpoint
jmx-exporter     scrapes localhost and exposes /metrics on Pod IP
Prometheus       scrapes Pod:9404 via ServiceMonitor/annotations
```

The network-namespace rule surprises people: `localhost` is shared across containers in a Pod, not private to each one.

7. Enterprise example

AxisPay uses the following sidecar split for card-processing services:

Service	Sidecar 1	Sidecar 2	Reason
payment-service	Fluent Bit	JMX exporter	Logs and JVM metrics are mandatory for live payments
fraud-service	Filebeat	JMX exporter	High event volume, custom log shipping pipeline
merchant-service	Fluent Bit	none	Lower JVM complexity; app exposes Micrometer directly

This means the Pod, not just the application JAR, is the deployable production unit.

8. Real-world analogy

A racing car and its support crew travelling in the same truck. The driver wins the race, but the telemetry engineer and mechanic are physically attached to the same operation because they must be present at the same place and time.

Where it breaks: a support crew can step away and still let the car race. In Kubernetes, if the Pod dies, all sidecars and the app die together.

9. Best practices

Practice	Reason
Give every sidecar explicit CPU and memory requests/limits	Prevents hidden resource theft and clearer scheduling
Use shared volumes only where needed	Reduces coupling and accidental data exposure
Prefer stdout logging unless a sidecar truly requires files	Simpler and more idiomatic for Kubernetes
Keep sidecars versioned independently from the app image	Platform concerns should not require app rebuilds
Document localhost port ownership inside the Pod	Avoids port collisions between containers
Monitor per-container memory, not only pod restarts	OOM root cause is often the sidecar
Evaluate native sidecars on Kubernetes 1.28+ for support containers that must start first	Better lifecycle modelling than ad hoc ordering

10. Common mistakes

Mistake	Symptom
Forgetting sidecar resource requests	Pod schedules too densely and later OOMs
Assuming localhost is private per container	Port conflicts or accidental cross-container access
Writing logs to a shared volume with no rotation	Disk pressure on the node
Bundling exporter logic into the app image	Unnecessary rebuilds for platform-only changes
Treating all sidecars as "free"	Performance overhead grows with every supporting container

11. Security considerations

Threat	Control	Residual risk
Sidecar can read app log volume containing sensitive data	Minimise logged PII; use redaction at source	Shared-volume readers are still trusted insiders
Shared localhost means sidecar can reach internal admin ports	Bind sensitive admin endpoints carefully or require auth	Same-Pod compromise has broad local visibility
Public sidecar image introduces supply-chain risk	Pin digests; scan images; use approved registries	Third-party code still expands the attack surface

12. Performance considerations

- Sidecars consume CPU during the same traffic spikes as the app. Log shippers are often busiest exactly when `payment-service` is busiest.
- File-based logging adds disk I/O compared with stdout-only logging.
- JMX scraping and translation are not free; polling too frequently increases JVM overhead.
- Pod density calculations must include sidecars. A "500Mi app" plus "150Mi logger" plus "100Mi exporter" is a 750Mi Pod, not a 500Mi Pod.
- Native sidecars can improve startup sequencing, but they do not remove the resource cost of the supporting container itself.

13. High availability

Sidecars improve operational availability when they standardise logging and metrics, because failures become visible faster. But they can reduce application availability if a failing sidecar causes the whole Pod to restart or exceed memory.

The HA rule is simple: supporting containers must be **less** fragile than the app they support.

14. Disaster recovery

In DR rebuilds, sidecars matter because they restore observability quickly. A recovered service that processes payments but emits no logs or metrics is operationally unsafe.

Treat sidecar configuration — parser configs, JMX rules, log routing — as versioned artefacts alongside the Deployment manifest.

15. Monitoring

Signal	Why
Per-container memory usage inside the Pod	Identifies whether app or sidecar caused OOM
Log shipping backlog / dropped records	Shows observability degradation before incident response does
Sidecar restart count	Failing support container often predicts wider Pod instability
JMX exporter scrape latency	Detects overloaded exporter or app JVM

16. Troubleshooting

Symptom	Likely cause	Command	Fix
Pod OOMKilled but app logs look normal	Logging or metrics sidecar consumed memory	<code>kubectl top pod --containers;</code> <code>kubectl describe pod</code>	Set realistic sidecar limits and requests
Prometheus shows no JVM metrics	Exporter sidecar not listening or wrong localhost target	<code>kubectl logs <pod> -c jmx-exporter;</code> <code>kubectl exec <pod> -c jmx-exporter -- wget -qO- localhost:9404/metrics</code>	Fix exporter config or port
Central logs missing for one Pod	Sidecar cannot read shared volume or output blocked	<code>kubectl logs <pod> -c fluent-bit</code>	Fix volume mount path or sink credentials
Pod starts, sidecar never does	Sidecar image pull or config error	<code>kubectl describe pod</code>	Repair sidecar image/config
Port 9404 already in use	Another container in the Pod bound the same port	<code>kubectl exec <pod> -c payment-service -- ss -lntp</code>	Reassign container ports clearly

Interview questions

- Why can a sidecar scrape `localhost` without a Kubernetes Service?** *Because containers in the same Pod share one network namespace and therefore one loopback interface. `localhost` is Pod-local, not container-local.*
- Why do sidecars need their own resource requests and limits?** *Because they consume real CPU and memory. If you omit them, the scheduler underestimates Pod size and later OOM or throttling is blamed unfairly on the application container.*
- When would you choose a sidecar instead of a library inside the Java app?** *When the concern is platform-owned and should evolve independently, such as log shipping, generic metrics exporting or policy enforcement.*
- What is the difference between the legacy sidecar pattern and native sidecars in Kubernetes 1.28+?** *(senior) Legacy sidecars are ordinary long-running containers started alongside the app. Native sidecars use `restartPolicy: Always` on an init-container-style definition so Kubernetes can better model startup ordering and lifecycle semantics for support containers that must start first and stay running.*
- What is the hidden failure mode of file-sharing sidecars?** *(senior) The shared volume becomes a coupling point: bad permissions, unexpected file growth, parser lag or disk pressure in the log shipper can all affect the whole Pod even though the business app code is correct.*

1.10 Deep Dive: AxisPay Service Architecture

Day 1 teaches objects in isolation: Pods, Deployments, Services, labels and reconciliation. Real systems are not isolated. A card payment flows across several services, and Kubernetes matters at every hop.

A normal AxisPay authorisation starts at the merchant client. The merchant sends a tokenised card payment request to the edge namespace, where `auth-service` first validates the caller's bearer token and merchant entitlements. The client does not talk to Pods directly; it talks to a stable Kubernetes Service. Behind that Service sits a Deployment of `auth-service` Pods. Rolling updates can replace individual Pods because the Service name stays fixed and only ready Pods become endpoints.

Once the request is authenticated, `auth-service` calls `payment-service` over its ClusterIP Service in `axispay-core`. This matters because Pod IPs change constantly. The Deployment behind `payment-service` can scale from three Pods to six under load without the caller changing configuration. The desired state is declarative: the team says how many replicas and what image should exist, and controllers keep making that true.

Inside `payment-service`, the request is enriched with merchant routing rules and feature flags. Those values are not hard-coded. They come from ConfigMaps so operations can, for example, disable a risky issuer route or enable a new idempotency behaviour without rebuilding the container image. Sensitive values — database passwords, HMAC signing keys, third-party API credentials — come from Secrets. This is the first Day 1 architectural lesson in practice: configuration and runtime identity live outside the container image, but are still managed declaratively.

Before `payment-service` asks the bank to reserve funds, it performs a synchronous call to `fraud-service`. That call runs on a tight timeout because fraud scoring is part of the user-facing path. If the fraud decision takes too long, the entire payment becomes slow. Kubernetes objects matter here too. `fraud-service` sits behind its own Service, backed by a Deployment. Readiness probes keep slow-starting or degraded Pods out of rotation. If one Pod dies, the Service keeps routing to the others. Self-healing is not a slogan here; it is what stops one Java crash from becoming a merchant outage.

If `fraud-service` returns a permissive score, `payment-service` moves to `core-service`, AxisPay's ledger service. `core-service` is where money becomes accounting. It writes the authorisation hold as a double-entry reservation: one ledger entry for the customer liability and one for the merchant-side pending settlement account. This step is intentionally synchronous. If the ledger cannot reserve the money, the payment must fail cleanly rather than drift into an uncertain state.

At this point several Kubernetes concepts meet:

Hop	Kubernetes object doing the stability work	Why it matters
Client → <code>auth-service</code>	Service + Deployment	Stable address while Pods roll
<code>auth-service</code> → <code>payment-service</code>	ClusterIP Service	Pod IP churn hidden from callers
<code>payment-service</code> → <code>fraud-service</code>	Service + readiness probes	Slow or broken Pods removed from traffic
<code>payment-service</code> → <code>core-service</code>	Service + Secrets + ConfigMaps	Stable routing plus controlled credentials/config
<code>core-service</code> → database	Pod + Secret + persistent data tier	Workload identity separated from secret material

After the ledger booking succeeds, the synchronous payment path is effectively complete: the merchant gets an authorisation response. But AxisPay still has follow-up work. `payment-service` emits an asynchronous event for `merchant-service`, which later sends a webhook notification back to the merchant platform. This is where Kubernetes helps in a different way. The webhook sender can roll independently from the card-authorisation path because the services are separate Deployments with separate desired state. One team can update merchant notification formatting without risking the ledger path.

Kubernetes configuration also shapes operational control around the flow. Feature toggles for fraud thresholds and issuer routing live in ConfigMaps so they can be reviewed and changed separately from the Java code. API keys for webhook signing and external fraud feeds live in Secrets so they are mounted consistently across replicas. Deployments give each service a controlled rollout boundary. Services give every caller a stable name. The result is that a single failed Pod rarely matters; what matters is whether the **set** of ready replicas behind each Service still matches the declared design.

Why is this architecture such a good Day 1 example? Because it demonstrates the value of Kubernetes before advanced features appear. Declarative desired state means the platform team says "three payment Pods and three fraud Pods should exist" and the system recreates them when a node dies. Services provide stable identity while those Pods are replaced. Rolling updates allow a new `payment-service` image to come online gradually, with readiness protecting live traffic. Self-healing is not only about crashes; it is also about replacing bad instances with no operator logging into a server.

The architecture also explains why lifecycle settings matter so much for Java. `payment-service` is not a stateless toy. If a Pod is killed mid-request, the blast radius reaches fraud checks, ledger reservations and merchant retries. That is why later sections add startup probes, graceful shutdown, init containers and sidecars: they are not optional decorations, but the controls that make this microservice chain safe under change.

In other words, Day 1 objects are simple individually but powerful in combination. A payment succeeds because several small Kubernetes abstractions — Pods, Deployments, Services, ConfigMaps and Secrets — each do one boring job reliably.

1.11 Troubleshooting Java Workloads: Real Incident Walkthroughs

1. What it is

This section is a practical guide to debugging Java workloads on Kubernetes using the cluster's own evidence: status, events, logs and container settings.

2. Why it exists

Java incidents often present indirectly. The symptom shown by Kubernetes is rarely the root cause. `CrashLoopBackOff` is not a cause. `ImagePullBackOff` is not a cause. `Running 0/1` is not a cause. They are wrappers around JVM, registry, dependency and configuration failures.

3. The business problem

AxisPay's most expensive outages were not caused by exotic Kubernetes bugs. They were caused by ordinary operational mismatches interpreted too slowly:

- a container memory limit lower than the JVM heap assumption
- a missing registry credential after rotating a pull secret
- a readiness probe tied to a database that was still recovering

Each one looked different on the surface. Each one became diagnosable only when engineers followed a strict sequence: **describe** → **logs** → **compare desired vs actual**.

4. How it works

For Java services, the fastest reliable workflow is:

1. Check pod phase and restart count.
2. Read Events with `kubectl describe pod`.
3. Read previous logs if the container is restarting.
4. Compare manifest limits, probes and image settings against the symptom.
5. Fix the configuration mismatch before touching code.

5. Internal architecture

Signal source	What it tells you
Pod status	Whether the container started, is running, and is ready
Events	Scheduler, kubelet and image-pull failures
<code>--previous</code> logs	Why the last Java process exited
Container resource limits	Whether the JVM fits the container
Probe configuration	Whether the platform is misjudging startup or readiness

6. Component interactions

During an incident, four layers are interacting:

```
registry / image pull
container runtime / kubelet
JVM / Spring Boot process
Kubernetes probes and Services
```

The art is identifying which layer failed first.

7. Enterprise example

AxisPay's on-call runbook for Java services says: do not guess from the high-level pod status. A `CrashLoopBackOff` might be OOM, bad config, failed migration or an application exception. A `Running` Pod can still be absent from the Service. Diagnosis starts from evidence, not from the name of the state.

8. Real-world analogy

A car dashboard warning light. "Engine" is not a diagnosis; it is a category. You still need to open the bonnet, read the fault code and check which subsystem failed.

Where it breaks: Kubernetes gives you better evidence than a car dashboard, but only if you look at the right object in the right order.

9. Best practices

Practice	Reason
Always use <code>kubectl logs --previous</code> for restarting Java containers	The live container may not exist long enough to inspect
Keep JVM flags visible in manifests or config	Hidden entrypoint defaults are hard to compare during incidents
Separate startup, readiness and liveness endpoints	Makes probe failures interpretable
Use descriptive image tags and pinned registries	Shortens pull-failure diagnosis
Alert on restart rate, not just total failures	Catches flapping before full outage

10. Common mistakes

Mistake	Symptom
Looking at current logs instead of <code>--previous</code>	Empty output during CrashLoop diagnosis
Increasing memory limit without changing JVM settings	OOM repeats because heap still overcommits native memory
Recreating pods before reading Events	Evidence disappears from immediate view
Treating readiness failure as a liveness problem	Unnecessary restarts worsen recovery

11. Security considerations

Threat	Control	Residual risk
Incident logs contain tokens or PAN-adjacent data	Redact at source; restrict log access	Emergency debugging still widens visibility
Debugging private registries exposes credentials in shell history	Use imagePullSecrets, not ad hoc <code>docker login</code> on nodes	Secret misconfiguration still causes pull failures
Over-broad <code>kubectl exec</code> during incidents	Restrict RBAC and prefer read-only commands first	Break-glass access remains sensitive

12. Performance considerations

- Repeated crash loops create load on the API server, kubelet and registry.
- Mis-tuned readiness probes can hold capacity out of rotation for minutes during a spike.
- OOM-restarting JVMs often thrash the node page cache and hurt neighbouring Pods.

13. High availability

The purpose of good troubleshooting is to restore healthy replicas before redundancy is exhausted. In a three-replica Deployment, one broken Pod is an incident warning. Two broken Pods is now an availability problem. Fast diagnosis protects HA.

14. Disaster recovery

These incidents are not classic DR events, but the same principle applies: preserve the evidence and keep the configuration in Git. If a cluster rebuild reuses the same bad image pull secret or the same oversized JVM heap, you have merely recreated the outage.

15. Monitoring

Signal	Why
Restart rate by Deployment	First sign of Java instability
OOM kill count	Detects heap/limit mismatch
Image pull failure events	Registry or credential issue
Readiness probe failure rate	Distinguishes degraded start from healthy service

16. Troubleshooting

Incident A — `CrashLoopBackOff` caused by `java.lang.OutOfMemoryError`

Narrative. After a release, one `payment-service` Pod never stayed up for more than 25 seconds. The Deployment was unchanged except for a new image. `kubectl get pods` showed `CrashLoopBackOff`, but the cause was inside the last terminated JVM.

```
kubectl logs payment-service-7d9b68d57d-lwckm --previous
```

```
2026-08-14 10:22:14.419 INFO 1 --- [           main] c.a.payment.PaymentApplication :
Starting PaymentApplication using Java 17
2026-08-14 10:22:28.804 INFO 1 --- [           main]
com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
Exception in thread "http-nio-8080-exec-7" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.util.Arrays.copyOf(Arrays.java:3537)
    at java.base/
java.lang.AbstractStringBuilder.ensureCapacityInternal(AbstractStringBuilder.java:228)
    at java.base/java.lang.StringBuilder.append(StringBuilder.java:179)
    at com.axispay.payment.audit.JsonAuditEncoder.encode(JsonAuditEncoder.java:88)
    at com.axispay.payment.audit.AuditService.write(AuditService.java:51)
```

The manifest had `resources.limits.memory: 768Mi`, but the container entrypoint still set `-Xmx1024m`. Even worse, native memory, metaspace and thread stacks sit **outside** the Java heap. The JVM was guaranteed to exceed the cgroup limit.

Fix. Lower the heap relative to the container limit, or let the JVM size itself from cgroup memory:

```
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=60 -XX:InitialRAMPercentage=30"
resources:
  requests:
    memory: 512Mi
  limits:
    memory: 768Mi
```

For teams that prefer explicit values, `-Xmx512m` would also fit. The important point is that heap must leave room for non-heap memory.

Incident B — `ImagePullBackOff` on the private fraud-service registry image

Narrative. A new `fraud-service` rollout created Pods that never started. Logs were useless because the container had never been created.

```
kubectl describe pod fraud-service-7854fc4979-19s9z
```

```
Events:
  Type      Reason      Age      From      Message
  ----      -
  Normal    Scheduled   33s      default-scheduler   Successfully assigned
axispay-core/fraud-service-7854fc4979-19s9z to axispay-m02
  Normal    Pulling     31s      kubelet    Pulling image
"registry.axispay.internal/fraud-service:1.14.2"
  Warning   Failed      30s      kubelet    Failed to pull image
"registry.axispay.internal/fraud-service:1.14.2": failed to authorize: unexpected status
from GET https://registry.axispay.internal/v2/token: 401 Unauthorized
  Warning   Failed      30s      kubelet    Error: ErrImagePull
  Normal    BackOff     18s (x3 over 29s) kubelet    Back-off pulling image
"registry.axispay.internal/fraud-service:1.14.2"
  Warning   Failed      18s (x3 over 29s) kubelet    Error: ImagePullBackOff
```

The root cause was an `imagePullSecrets` reference to `regcred-prod`, but the secret in `axispay-core` had been created as `registry-cred-prod` during a credential rotation.

Fix. Correct the secret name in the Pod template and ensure the secret exists in the same namespace:

```
spec:
  imagePullSecrets:
    - name: registry-cred-prod
```

Then verify:

```
kubectl get secret registry-cred-prod -n axispay-core
kubectl rollout restart deploy/fraud-service -n axispay-core
```

Incident C — Pod `Running` but failing readiness forever

Narrative. `core-service` showed `Running 0/1` for nearly ten minutes after a node replacement. Operators first suspected slow startup, but `describe` showed a different story.

```
kubectl describe pod core-service-5ff88c7599-6gm7z
```

```
Events:
  Type      Reason      Age           From          Message
  ----      -
  Warning   Unhealthy   8m12s (x96 over 9m)   kubelet       Readiness probe failed: Get "http://10.244.1.42:8080/readyz": dial tcp 10.244.1.42:8080: connect: connection refused
  Warning   Unhealthy   5m40s (x44 over 7m)   kubelet       Readiness probe failed: HTTP probe failed with statuscode: 503
```

Application logs showed Spring Boot starting normally, then waiting on a database health indicator because PostgreSQL was still replaying WAL after failover. The Pod was **correctly** unready, but without a `startupProbe`, the team could not easily distinguish "still booting" from "booted but dependency unavailable".

Fix. Add a generous `startupProbe` so liveness and readiness are not evaluated as if boot and dependency recovery are the same phase:

```
startupProbe:
  httpGet:
    path: /startupz
    port: 8080
  periodSeconds: 5
  failureThreshold: 24
readinessProbe:
  httpGet:
    path: /readyz
    port: 8080
  periodSeconds: 5
  failureThreshold: 3
```

This gives the app up to two minutes to finish real startup. After that, continuing readiness failure is more likely to represent a broken dependency path than simple class loading.

Decision rule. If logs show the web server never bound the port, think startup. If the port is open but readiness returns 503 because a downstream dependency is red, think dependency health — do not add an aggressive liveness probe and make it worse.

Interview questions

1. **Why is** `kubectl logs --previous` **essential for** `CrashLoopBackOff` ? *Because the currently running container may be brand new or may not stay alive long enough to emit the failure. `--previous` shows the last terminated instance, which usually contains the real exception.*

2. Why can a JVM OOM even when `-Xmx` is lower than the container memory limit?

Because heap is only part of process memory. Metaspace, direct buffers, thread stacks, JIT code cache and native libraries all consume memory outside the heap.

3. Why are logs useless for `ImagePullBackOff` ?

Because the container never started. The evidence is in Pod Events from the kubelet, especially authorization, DNS or tag errors while pulling the image.

4. How do you distinguish slow startup from permanent readiness failure? (senior)

Correlate Events with application logs. If the server port never opens and startup logs are incomplete, it is startup. If the app starts but readiness returns 503 because a downstream dependency is unhealthy, it is a serving-readiness problem. `startupProbe` separates those phases cleanly.

5. What is the safest JVM sizing rule inside containers? (senior)

Size the heap as a fraction of cgroup memory using `-XX:MaxRAMPercentage` or a conservatively chosen explicit `-Xmx`, leaving room for non-heap memory. Then verify with real memory telemetry rather than trusting defaults.

Day 1 cheat sheet

The 6-step triage loop

```

1. DESIRED state?      kubectl get <kind> <name> -o yaml
2. ACTUAL state?      kubectl get pods -o wide
3. CLUSTER says?      kubectl describe <kind> <name>
                        kubectl get events --sort-by='.lastTimestamp' | tail -20
4. APP says?          kubectl logs <pod> [--previous] [-c <container>]
5. From INSIDE?       kubectl exec -it <pod> -- sh
                        kubectl debug -it <pod> --image=busybox
                        kubectl port-forward <pod> 8080:8080
6. What CHANGED?      kubectl rollout history deployment/<name>
                        git log --oneline -10

```

FIX → VERIFY → what would have caught this first?

Pod status decision table

Status	Container started?	First move
Pending	No — not even placed	describe → Events
ContainerCreating	No — being set up	describe → image pull or volume
ImagePullBackOff	Never	describe → Events. Logs are useless.
CrashLoopBackOff	Yes, then exited	logs --previous
Running 0/1	Yes, not serving	Readiness probe / dependency
Running 1/1	Yes, healthy	Look higher: Service, DNS, Ingress
Terminating (stuck)	Shutting down	Finaliser or grace period

Commands you will use every day

```
# context and scope
kubectl config use-context axispay
kubectl config set-context --current --namespace=axispay-core

# see everything AxisPay
kubectl get all -A -l app.kubernetes.io/part-of=axispay
kubectl get pods -A -o wide

# what did day 3 add?
kubectl get all -A -l axispay.io/day-introduced=3

# the four that solve most problems
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by='.lastTimestamp' | tail -20
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

# built-in documentation - offline
kubectl explain deployment.spec.strategy
kubectl api-resources --namespaced=false

# reach a pod from your laptop
kubectl port-forward -n axispay-core deploy/payment-service 8083:8080

# throwaway debug pod inside the cluster
kubectl run dbg -n axispay-edge --rm -it --restart=Never \
  --image=curlimages/curl:8.11.1 -- sh
```

AxisPay conventions

Thing	Format	Example
Merchant ID	MER_ + 10 chars	MER_7QK2XD9P4A
Payment ID	pay_ + 24 hex	pay_9f2c41ab77de0c3518be4d6a
Reference	AXP-YYYYMMDD- + 8 hex	AXP-20260810-4c9a1f77
Card token	tok_ + 24 hex	tok_a71ef4c2900bd5386ff1240e
Money	Integer minor units + ISO-4217	129900 ZAR = R1,299.00

Every service exposes: /healthz · /readyz · /startupz · /metrics · /api/v1/_info

Day 1 review questions

1. What are the two states every Kubernetes object has, and who writes each?
2. Trace what happens when you delete one pod of a 3-replica Deployment.
3. Name every control-plane component and give each a one-sentence responsibility.
4. Does the scheduler start containers?
5. Do namespaces isolate network traffic? Justify your answer.
6. Name three cluster-scoped resources.
7. Why does the Pod exist rather than scheduling containers directly?
8. What does the `pause` container do?
9. A pod is `Running` but `0/1`. What does that mean, and what is your first command?
10. What owns a Pod created by a Deployment, and why does that matter?
11. Why is `spec.selector` immutable?
12. Why does Kubernetes retain old ReplicaSets at zero replicas?
13. A Service has a ClusterIP but requests fail. What is your first command?
14. How does a Service decide which pods receive traffic?
15. Why is there no proxy process in a ClusterIP request path?
16. `kubectl logs` says "container is waiting to start". What is happening and what do you do?
17. How do you distinguish `ImagePullBackOff` from `CrashLoopBackOff` from the output of one command?
18. What single idea underlies every controller in Kubernetes?

Answers: `documents/assessments/answer-keys/day1-answer-key.md`

Day 1 summary

You built: three labelled namespaces · four Deployments · four Services · nine pods across two namespaces · one real payment processed end-to-end with correct fee arithmetic and working idempotency.

You learned: the declarative model and the reconciliation loop · cluster architecture and the hub-and-spoke communication pattern · namespaces as trust boundaries · Pods and why bare ones are unfit for production · Deployments, ReplicaSets and self-healing · Services, label selection and load balancing · a repeatable triage method, applied under pressure to a real incident.

What is still missing — and when you fix it:

Gap	Consequence	Fixed
No resource requests	The scheduler is guessing; pods land badly and get OOM-killed	L2.1
No probes	Kubernetes cannot tell "started" from "able to serve" — today's incident took out all three replicas	L2.3
No autoscaling	Black Friday is a manual <code>kubect1 scale</code> at 2am	L2.4
No graceful shutdown	A rolling update severs in-flight payments	L2.6
Nothing scheduled	Settlement at 23:00 is a cron job on someone's laptop	L2.5
Payments vanish on restart	All state is in memory	Day 3
Config and signing key in plain env vars	Visible to anyone with namespace read access	Day 3
Unreachable from outside the cluster	No merchant can actually integrate	Day 4
Edge can reach the data tier	A PCI finding	Day 4
No RBAC, metrics, dashboards or alerts	You cannot operate what you cannot see	Day 5

Tonight (optional, 20 minutes): run `kubect1 explain deployment.spec.strategy` and read it. Tomorrow starts there.